ARTICLE

doi:10.1038/nature16961

Mastering the game of Go with deep neural networks and tree search

David Silver[1]*, Aja Huang[1]*, Chris J. Maddison[1], Arthur Guez[1], Laurent Sifre[1], George van den Driessche[1], Julian Schrittwieser[1], Ioannis Antonoglou[1], Veda Panneershelvam[1], Marc Lanctot[1], Sander Dieleman[1], Dominik Grewe[1], John Nham[2], Nal Kalchbrenner[1], Ilya Sutskever[2], Timothy Lillicrap[1], Madeleine Leach[1], Koray Kavukcuoglu[1], Thore Graepel[1] & Demis Hassabis[1]

The game of Go has long been viewed as the most challenging of classic games for artificial intelligence owing to its enormous search space and the difficulty of evaluating board positions and moves. Here we introduce a new approach to computer Go that uses 'value networks' to evaluate board positions and 'policy networks' to select moves. These deep neural networks are trained by a novel combination of supervised learning from human expert games, and reinforcement learning from games of self-play. Without any lookahead search, the neural networks play Go at the level of state-of-the-art Monte Carlo tree search programs that simulate thousands of random games of self-play. We also introduce a new search algorithm that combines Monte Carlo simulation with value and policy networks. Using this search algorithm, our program AlphaGo achieved a 99.8% winning rate against other Go programs, and defeated the human European Go champion by 5 games to 0. This is the first time that a computer program has defeated a human professional player in the full-sized game of Go, a feat previously thought to be at least a decade away.

All games of perfect information have an optimal value function, $v^*(s)$, which determines the outcome of the game, from every board position or state s, under perfect play by all players. These games may be solved by recursively computing the optimal value function in a search tree containing approximately b^d possible sequences of moves, where b is the game's breadth (number of legal moves per position) and d is its depth (game length). In large games, such as chess ($b \approx 35$, $d \approx 80$)[1] and especially Go ($b \approx 250$, $d \approx 150$)[1], exhaustive search is infeasible[2,3], but the effective search space can be reduced by two general principles. First, the depth of the search may be reduced by position evaluation: truncating the search tree at state s and replacing the subtree below s by an approximate value function $v(s) \approx v^*(s)$ that predicts the outcome from state s. This approach has led to superhuman performance in chess[4], checkers[5] and othello[6], but it was believed to be intractable in Go due to the complexity of the game[7]. Second, the breadth of the search may be reduced by sampling actions from a policy $p(a|s)$ that is a probability distribution over possible moves a in position s. For example, Monte Carlo rollouts[8] search to maximum depth without branching at all, by sampling long sequences of actions for both players from a policy p. Averaging over such rollouts can provide an effective position evaluation, achieving superhuman performance in backgammon[8] and Scrabble[9], and weak amateur level play in Go[10].

Monte Carlo tree search (MCTS)[11,12] uses Monte Carlo rollouts to estimate the value of each state in a search tree. As more simulations are executed, the search tree grows larger and the relevant values become more accurate. The policy used to select actions during search is also improved over time, by selecting children with higher values. Asymptotically, this policy converges to optimal play, and the evaluations converge to the optimal value function[12]. The strongest current Go programs are based on MCTS, enhanced by policies that are trained to predict human expert moves[13]. These policies are used to narrow the search to a beam of high-probability actions, and to sample actions during rollouts. This approach has achieved strong amateur play[13–15]. However, prior work has been limited to shallow policies[13–15] or value functions[16] based on a linear combination of input features.

Recently, deep convolutional neural networks have achieved unprecedented performance in visual domains: for example, image classification[17], face recognition[18], and playing Atari games[19]. They use many layers of neurons, each arranged in overlapping tiles, to construct increasingly abstract, localized representations of an image[20]. We employ a similar architecture for the game of Go. We pass in the board position as a 19×19 image and use convolutional layers to construct a representation of the position. We use these neural networks to reduce the effective depth and breadth of the search tree: evaluating positions using a value network, and sampling actions using a policy network.

We train the neural networks using a pipeline consisting of several stages of machine learning (Fig. 1). We begin by training a supervised learning (SL) policy network p_σ directly from expert human moves. This provides fast, efficient learning updates with immediate feedback and high-quality gradients. Similar to prior work[13,15], we also train a fast policy p_π that can rapidly sample actions during rollouts. Next, we train a reinforcement learning (RL) policy network p_ρ that improves the SL policy network by optimizing the final outcome of games of self-play. This adjusts the policy towards the correct goal of winning games, rather than maximizing predictive accuracy. Finally, we train a value network v_θ that predicts the winner of games played by the RL policy network against itself. Our program AlphaGo efficiently combines the policy and value networks with MCTS.

Supervised learning of policy networks

For the first stage of the training pipeline, we build on prior work on predicting expert moves in the game of Go using supervised learning[13,21–24]. The SL policy network $p_\sigma(a|s)$ alternates between convolutional layers with weights σ, and rectifier nonlinearities. A final softmax layer outputs a probability distribution over all legal moves a. The input s to the policy network is a simple representation of the board state (see Extended Data Table 2). The policy network is trained on randomly

[1]Google DeepMind, 5 New Street Square, London EC4A 3TW, UK. [2]Google, 1600 Amphitheatre Parkway, Mountain View, California 94043, USA.
*These authors contributed equally to this work.

© 2016 Macmillan Publishers Limited. All rights reserved

文章

doi:10.1038/nature16961

利用深度神经网络和树搜索技术掌握围棋

David Silver*、Aja Huang*、Chris J. Maddison、Arthur Guez、Laurent Sifre、George van den Driessche、Julian Schrittwieser、Ioannis Antonoglou、Veda Panneershelvam、Marc Lanctot、Sander Dieleman、Dominik Grewe、John Nham、Nal Kalchbrenner、Ilya Sutskever、Timothy Lillicrap、Madeleine Leach、Koray Kavukcuoglu、托雷·格雷佩尔和黛米斯·哈萨比斯

由于围棋庞大的搜索空间以及评估棋局和走法的难度，围棋长期以来被视为人工智能最具挑战性的经典游戏之一。本文提出了一种新的计算机围棋方法，该方法使用“价值网络”评估棋局，并使用“策略网络”选择走法。这些深度神经网络通过一种新颖的组合进行训练：一方面是来自人类专家对局的监督学习，另一方面是来自自我对弈的强化学习。无需任何前瞻搜索，这些神经网络的围棋水平就达到了最先进的蒙特卡洛树搜索程序的水平，后者能够模拟数千局随机的自我对弈。我们还提出了一种新的搜索算法，该算法将蒙特卡洛模拟与价值网络和策略网络相结合。利用这种搜索算法，我们的程序 AlphaGo 在与其他围棋程序的对弈中取得了 99.8%的胜率，并以 5 比 0 的比分击败了欧洲围棋冠军。这是计算机程序首次在完整的围棋比赛中击败人类职业棋手，这一壮举此前被认为至少还需要十年才能实现。

所有完全信息博弈都存在一个最优值函数 v(s)，它决定了在所有玩家都完美下棋的情况下，从每个棋盘位置或状态 s 开始的博弈结果。这些博弈可以通过递归地计算最优值函数来解决，搜索树包含大约 b 个可能的走法序列，其中 b 是博弈的宽度（每个位置的合法走法数量），d 是博弈的深度（博弈长度）。在大型博弈中，例如国际象棋（b ≈ 35，d ≈ 80）和围棋（b ≈ 250，d ≈ 150），穷举搜索是不可行的，但可以通过两个一般原则来缩小有效搜索空间。首先，可以通过位置评估来减少搜索深度：在状态 s 处截断搜索树，并将 s 以下的子树替换为一个近似值函数 v(s) ≈ v(s)，该函数可以预测从状态 s 开始的博弈结果。这种方法在国际象棋、跳棋和奥赛罗棋中取得了超人般的成绩，但由于围棋的复杂性，人们认为这种方法在围棋中难以实现。其次，可以通过从策略 p(a|s) 中采样动作来缩小搜索范围，其中策略 p(a|s) 是位置 s 中所有可能走法 a 的概率分布。例如，蒙特卡洛滚动搜索通过从策略 p 中为双方玩家采样长序列的动作，在完全不进行分支的情况下搜索到最大深度。对这样的滚动搜索结果进行平均可以提供有效的位置评估，从而在西洋双陆棋和拼字游戏中达到超人般的水平，并在围棋中达到业余水平。

蒙特卡洛树搜索（MCTS）利用蒙特卡洛模拟来估计搜索树中每个状态的价值。随着模拟次数的增加，搜索树不断增长，相关价值也变得更加精确。用于在搜索过程中选择动作的策略也会随着时间的推移而改进，通过选择价值更高的子节点来实现。渐近地，该策略会收敛到最优解，评估结果也会收敛到最优值函数。目前最强大的围棋程序都基于 MCTS，并辅以经过训练以预测人类专家走法的策略。这些策略用于将搜索范围缩小到一组高概率动作，并在模拟过程中对动作进行采样。这种方法已经取得了很高的业余水平。然而以往的研究仅限于浅层策略或基于输入特征线性组合的价值函数。

近年来，深度卷积神经网络在视觉领域取得了前所未有的性能，例如图像分类、人脸识别和雅达利游戏。它们使用多层神经元，每层神经元排列成重叠的图块，从而构建图像越来越抽象、局部化的表示。我们采用类似的架构来处理围棋。我们将棋盘位置以 19 × 19 的图像形式输入，并使用卷积层构建该位置的表示。我们利用这些神经网络来降低搜索树的有效深度和广度：使用值网络评估位置，并使用策略网络对动作进行采样。

我们使用包含多个机器学习阶段的流水线来训练神经网络（图 1）。首先，我们直接利用专家棋手的走法训练一个监督学习（SL）策略网络 p。这能够提供快速高效的学习更新，并带来即时反馈和高质量的梯度。与之前的工作类似，我们也训练了一个快速策略 p，它可以在回合展开过程中快速采样动作。接下来，我们训练一个强化学习（RL）策略网络 p，它通过优化自我对弈的最终结果来改进 SL 策略网络。这使得策略朝着赢得比赛的正确目标进行调整，而不是最大化预测准确率。最后，我们训练一个价值网络 v，用于预测 RL 策略网络与自身对弈的胜负。我们的程序 AlphaGo 将策略网络和价值网络与蒙特卡洛树搜索（MCTS）高效地结合起来。

政策网络的监督学习

在训练流程的第一阶段，我们基于先前使用监督学习预测围棋专家走法的研究。SL 策略网络 p(a|s)交替使用权重为σ的卷积层和整流器非线性层。最终的 softmax 层输出所有合法走法 a 的概率分布。策略网络的输入 s 是棋盘状态的简单表示（参见扩展数据表 2）。策略网络在随机数据集上进行训练。

1 Google DeepMind，5 New Street Square，伦敦 EC4A 3TW，英国。Google，1600 Amphitheatre Parkway，Mountain View，加利福尼亚州 94043，美国。

*这些作者对本文贡献相同。

© 2016 麦克米伦出版有限公司。版权所有。

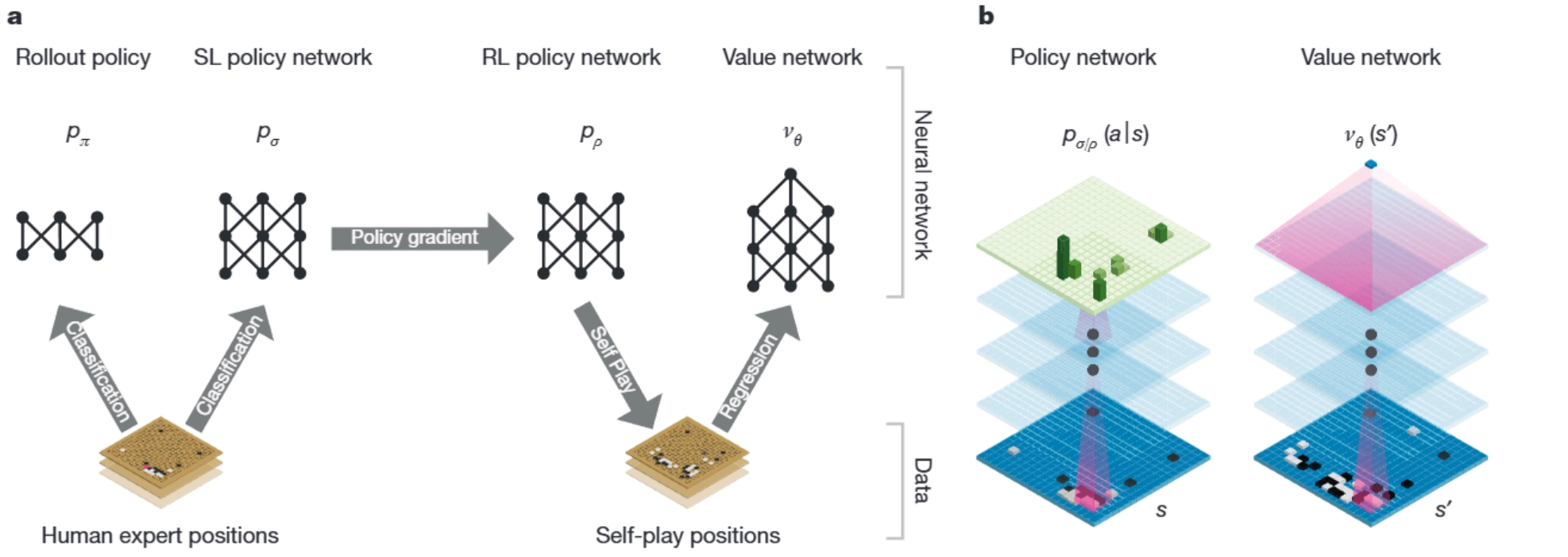


Figure 1 | Neural network training pipeline and architecture. **a**, A fast rollout policy p_π and supervised learning (SL) policy network p_σ are trained to predict human expert moves in a data set of positions. A reinforcement learning (RL) policy network p_ρ is initialized to the SL policy network, and is then improved by policy gradient learning to maximize the outcome (that is, winning more games) against previous versions of the policy network. A new data set is generated by playing games of self-play with the RL policy network. Finally, a value network v_θ is trained by regression to predict the expected outcome (that is, whether the current player wins) in positions from the self-play data set. **b**, Schematic representation of the neural network architecture used in AlphaGo. The policy network takes a representation of the board position s as its input, passes it through many convolutional layers with parameters σ (SL policy network) or ρ (RL policy network), and outputs a probability distribution $p_\sigma(a|s)$ or $p_\rho(a|s)$ over legal moves a, represented by a probability map over the board. The value network similarly uses many convolutional layers with parameters θ, but outputs a scalar value $v_\theta(s')$ that predicts the expected outcome in position s'.

sampled state-action pairs (s, a), using stochastic gradient ascent to maximize the likelihood of the human move a selected in state s

$$\Delta\sigma \propto \frac{\partial \log p_\sigma(a|s)}{\partial \sigma}$$

We trained a 13-layer policy network, which we call the SL policy network, from 30 million positions from the KGS Go Server. The network predicted expert moves on a held out test set with an accuracy of 57.0% using all input features, and 55.7% using only raw board position and move history as inputs, compared to the state-of-the-art from other research groups of 44.4% at date of submission[24] (full results in Extended Data Table 3). Small improvements in accuracy led to large improvements in playing strength (Fig. 2a); larger networks achieve better accuracy but are slower to evaluate during search. We also trained a faster but less accurate rollout policy $p_\pi(a|s)$, using a linear softmax of small pattern features (see Extended Data Table 4) with weights π; this achieved an accuracy of 24.2%, using just 2 µs to select an action, rather than 3 ms for the policy network.

Reinforcement learning of policy networks

The second stage of the training pipeline aims at improving the policy network by policy gradient reinforcement learning (RL)[25,26]. The RL policy network p_ρ is identical in structure to the SL policy network, and its weights ρ are initialized to the same values, $\rho = \sigma$. We play games between the current policy network p_ρ and a randomly selected previous iteration of the policy network. Randomizing from a pool of opponents in this way stabilizes training by preventing overfitting to the current policy. We use a reward function $r(s)$ that is zero for all non-terminal time steps $t < T$. The outcome $z_t = \pm r(s_T)$ is the terminal reward at the end of the game from the perspective of the current player at time step t: $+1$ for winning and -1 for losing. Weights are then updated at each time step t by stochastic gradient ascent in the direction that maximizes expected outcome[25]

$$\Delta\rho \propto \frac{\partial \log p_\rho(a_t|s_t)}{\partial \rho} z_t$$

We evaluated the performance of the RL policy network in game play, sampling each move $a_t \sim p_\rho(\cdot|s_t)$ from its output probability distribution over actions. When played head-to-head, the RL policy network won more than 80% of games against the SL policy network. We also tested against the strongest open-source Go program, Pachi[14], a sophisticated Monte Carlo search program, ranked at 2 amateur *dan* on KGS, that executes 100,000 simulations per move. Using no search at all, the RL policy network won 85% of games against Pachi. In comparison, the previous state-of-the-art, based only on supervised

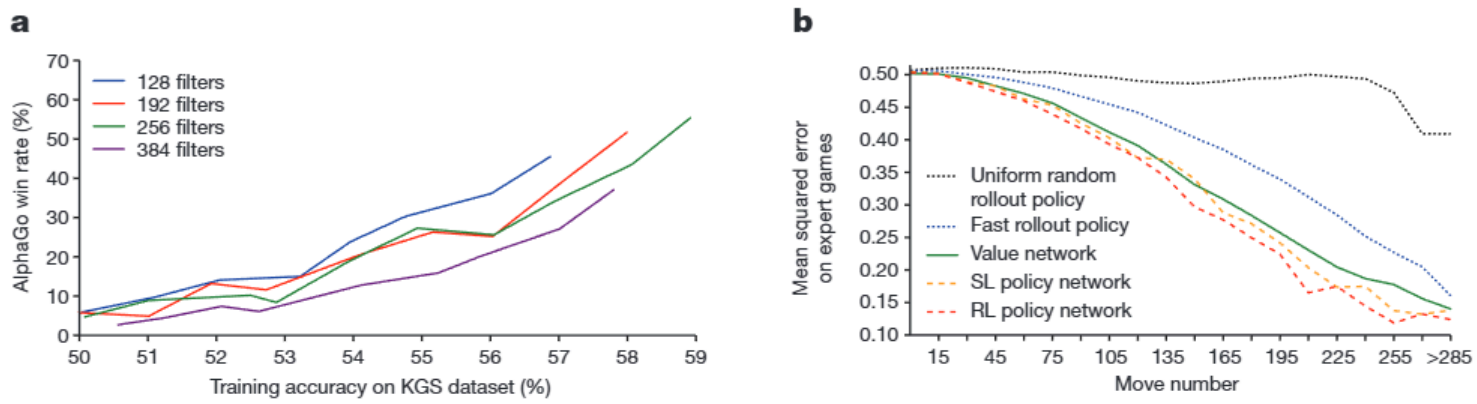


Figure 2 | Strength and accuracy of policy and value networks. **a**, Plot showing the playing strength of policy networks as a function of their training accuracy. Policy networks with 128, 192, 256 and 384 convolutional filters per layer were evaluated periodically during training; the plot shows the winning rate of AlphaGo using that policy network against the match version of AlphaGo. **b**, Comparison of evaluation accuracy between the value network and rollouts with different policies. Positions and outcomes were sampled from human expert games. Each position was evaluated by a single forward pass of the value network v_θ, or by the mean outcome of 100 rollouts, played out using either uniform random rollouts, the fast rollout policy p_π, the SL policy network p_σ or the RL policy network p_ρ. The mean squared error between the predicted value and the actual game outcome is plotted against the stage of the game (how many moves had been played in the given position).

© 2016 Macmillan Publishers Limited. All rights reserved

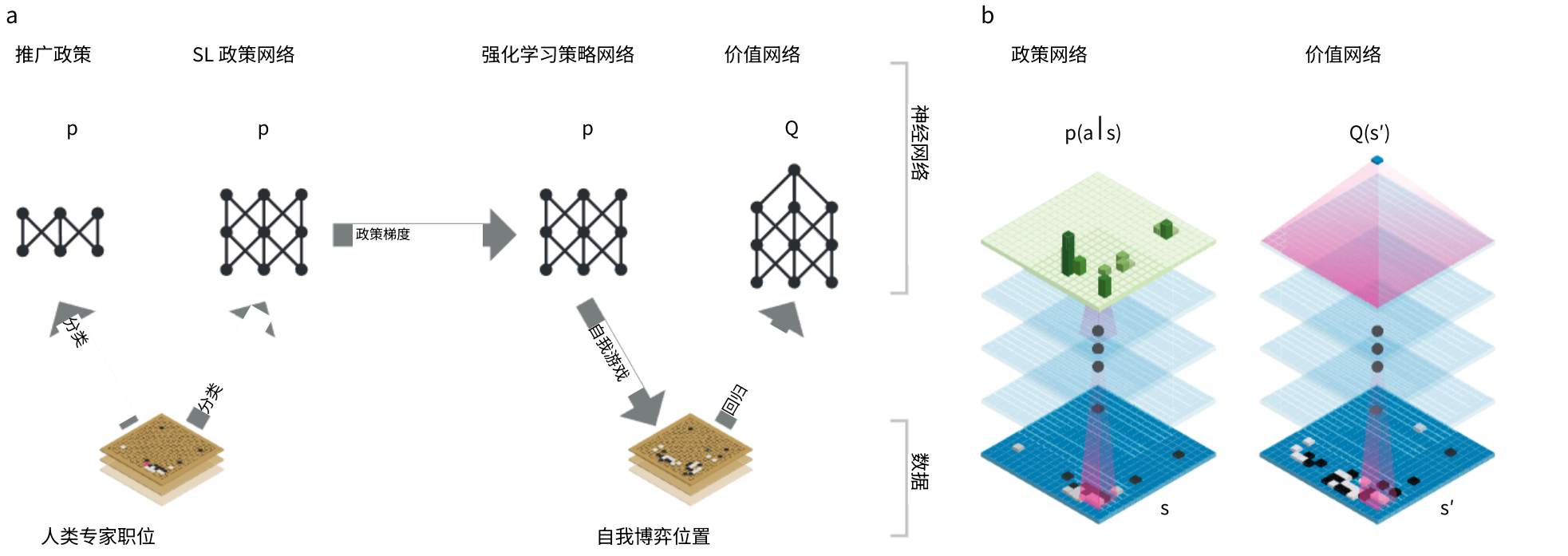


图 1 | 神经网络训练流程和架构。a，快速展开策略网络 p 和监督学习 (SL) 策略网络 par 经过训练，能够预测人类专家在一组棋局中的表现。强化学习 (RL) 策略网络 p 初始化为 SL 策略网络，然后通过策略梯度学习进行改进，以最大化其相对于先前版本策略网络的胜率（即赢得更多对局）。通过与 RL 策略网络进行自对弈，生成一个新的数据集。最后，通过回归训练一个价值网络 v，以预测预期结果（即是否获胜）。

使用随机梯度上升法对状态-动作对 (s, a) 进行采样，以最大化在状态 s 中选定的人类动作 a 的似然性。

$$\Delta\sigma \propto \frac{\partial \log p_\sigma(a|s)}{\partial \sigma}$$

我们使用来自 KGS 围棋服务器的 3000 万个棋局数据训练了一个 13 层的策略网络，我们称之为 SL 策略网络。该网络在预留的测试集上预测专家走法的准确率达到 57.0%（使用所有输入特征），仅使用原始棋盘位置和走棋历史作为输入时，准确率达到 55.7%。相比之下，其他研究团队在提交时的最先进水平为 44.4%（完整结果见扩展数据表 3）。准确率的微小提升带来了棋力的显著提高（图 2a）；更大的网络虽然能获得更高的准确率，但在搜索过程中评估速度较慢。我们还训练了一个速度更快但准确率较低的滚动策略 $p(a|s)$，该策略使用权重为 π 的线性 softmax 函数对小模式特征进行处理（见扩展数据表 4），准确率达到 24.2%，选择动作仅需 2 微秒，而策略网络则需要 3 毫秒。

策略网络的强化学习

训练流程的第二阶段旨在通过策略梯度强化学习（RL）改进策略网络。RL 策略网络的结构与 SL 策略网络相同。

a，当前玩家获胜）在自对弈数据集中的局面。b，AlphaGo 中使用的神经网络架构示意图。策略网络以棋盘局面 s 的表示作为输入，将其传递给多个卷积层，参数分别为 σ（SL 策略网络）或 ρ（RL 策略网络），并输出概率分布 $p_\sigma(a|s)$ 或 $p_\rho(a|s)$。

遍历所有合法走法 a，用棋盘上的概率图表示。价值网络类似地使用多个参数为 θ 的卷积层，但输出一个标量值 $v_\theta(s')$，用于预测位置 s′ 的预期结果。

其权重 ρ 初始化为相同的值，即 ρ = σ。我们让当前策略网络 p 与随机选择的策略网络的前一次迭代进行博弈。通过这种方式从一组对手中随机选择，可以防止过度拟合当前策略，从而稳定训练过程。我们使用奖励函数 r(s)，该函数在所有非终止时间步 t < T 时均为零。结果 $z_t = \pm r(s_T)$ 是当前玩家在时间步 t 的博弈结束时的最终奖励：+1 表示获胜，-1 表示失败。然后，在每个时间步 t，通过随机梯度上升法，沿着最大化预期结果的方向更新权重。

$$\Delta\rho \propto \frac{\partial \log p_\rho(a_t|s_t)}{\partial \rho} z_t$$

我们评估了 RL 策略网络在游戏过程中的性能，对每一步都进行了采样 $a_t \sim p_\rho(\cdot|s_t)$，来自其对动作的输出概率分布。在直接对弈中，强化学习策略网络 (RL) 战胜了策略学习 (SL) 策略网络，胜率超过 80%。我们还测试了最强大的开源围棋程序 Pachi，这是一个复杂的蒙特卡洛搜索程序，在 KGS 业余棋手排名中达到 2 段，每走一步棋都要进行 10 万次模拟。在完全不使用搜索的情况下，强化学习策略网络战胜了 Pachi，胜率达到 85%。相比之下，之前的最先进方法仅基于监督学习，

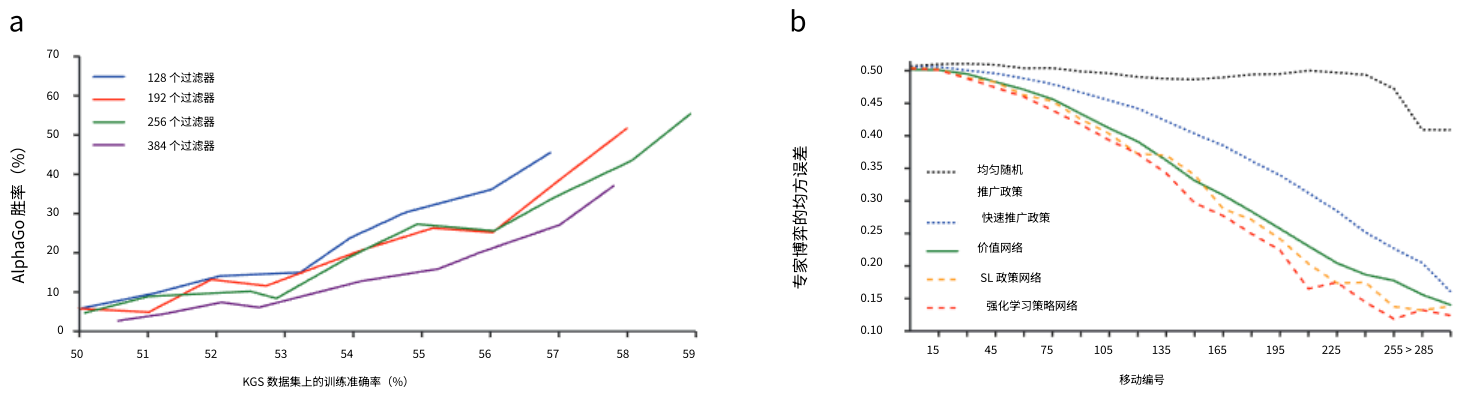


图 2 | 政策和价值网络的强度和准确性。a，图表展示了策略网络的对弈强度与其训练准确率的关系。在训练过程中，我们定期评估了每层分别包含 128、192、256 和 384 个卷积滤波器的策略网络；图表显示了 AlphaGo 使用这些策略网络对抗其比赛版本的胜率。b，比较了不同策略下的价值网络和展开式网络的评估准确率。棋局位置和结果均取自人类专家对弈。每个棋局位置均通过价值网络 v 的单次前向传播进行评估，或通过 100 次轮次推演的平均结果进行评估。这些轮次推演分别采用均匀随机推演、快速推演策略 p、SL 策略网络或 RL 策略网络 p。预测值与实际对弈结果之间的均方误差与对弈阶段（给定位置已进行的步数）的关系图被绘制出来。

© 2016 麦克米伦出版有限公司。版权所有。

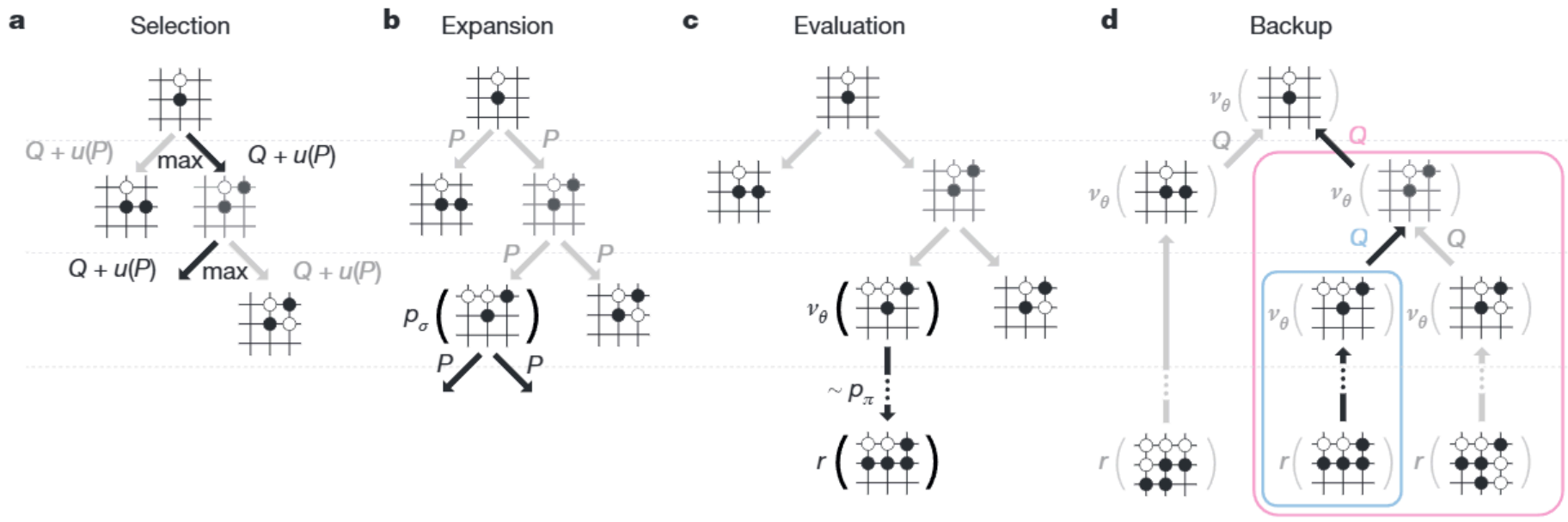


Figure 3 | Monte Carlo tree search in AlphaGo. a, Each simulation traverses the tree by selecting the edge with maximum action value Q, plus a bonus $u(P)$ that depends on a stored prior probability P for that edge. **b**, The leaf node may be expanded; the new node is processed once by the policy network p_σ and the output probabilities are stored as prior probabilities P for each action. **c**, At the end of a simulation, the leaf node is evaluated in two ways: using the value network v_θ; and by running a rollout to the end of the game with the fast rollout policy p_π, then computing the winner with function r. **d**, Action values Q are updated to track the mean value of all evaluations $r(\cdot)$ and $v_\theta(\cdot)$ in the subtree below that action.

learning of convolutional networks, won 11% of games against Pachi[23] and 12% against a slightly weaker program, Fuego[24].

Reinforcement learning of value networks

The final stage of the training pipeline focuses on position evaluation, estimating a value function $v^p(s)$ that predicts the outcome from position s of games played by using policy p for both players[28–30]

$$v^p(s) = \mathbb{E}[z_t | s_t = s,\ a_{t\ldots T} \sim p]$$

Ideally, we would like to know the optimal value function under perfect play $v^*(s)$; in practice, we instead estimate the value function v^{p_ρ} for our strongest policy, using the RL policy network p_ρ. We approximate the value function using a value network $v_\theta(s)$ with weights θ, $v_\theta(s) \approx v^{p_\rho}(s) \approx v^*(s)$. This neural network has a similar architecture to the policy network, but outputs a single prediction instead of a probability distribution. We train the weights of the value network by regression on state-outcome pairs (s, z), using stochastic gradient descent to minimize the mean squared error (MSE) between the predicted value $v_\theta(s)$, and the corresponding outcome z

$$\Delta\theta \propto \frac{\partial v_\theta(s)}{\partial\theta}(z - v_\theta(s))$$

The naive approach of predicting game outcomes from data consisting of complete games leads to overfitting. The problem is that successive positions are strongly correlated, differing by just one stone, but the regression target is shared for the entire game. When trained on the KGS data set in this way, the value network memorized the game outcomes rather than generalizing to new positions, achieving a minimum MSE of 0.37 on the test set, compared to 0.19 on the training set. To mitigate this problem, we generated a new self-play data set consisting of 30 million distinct positions, each sampled from a separate game. Each game was played between the RL policy network and itself until the game terminated. Training on this data set led to MSEs of 0.226 and 0.234 on the training and test set respectively, indicating minimal overfitting. Figure 2b shows the position evaluation accuracy of the value network, compared to Monte Carlo rollouts using the fast rollout policy p_π; the value function was consistently more accurate. A single evaluation of $v_\theta(s)$ also approached the accuracy of Monte Carlo rollouts using the RL policy network p_ρ, but using 15,000 times less computation.

Searching with policy and value networks

AlphaGo combines the policy and value networks in an MCTS algorithm (Fig. 3) that selects actions by lookahead search. Each edge (s, a) of the search tree stores an action value $Q(s, a)$, visit count $N(s, a)$, and prior probability $P(s, a)$. The tree is traversed by simulation (that is, descending the tree in complete games without backup), starting from the root state. At each time step t of each simulation, an action a_t is selected from state s_t

$$a_t = \underset{a}{\text{argmax}}(Q(s_t, a) + u(s_t, a))$$

so as to maximize action value plus a bonus

$$u(s, a) \propto \frac{P(s, a)}{1 + N(s, a)}$$

that is proportional to the prior probability but decays with repeated visits to encourage exploration. When the traversal reaches a leaf node s_L at step L, the leaf node may be expanded. The leaf position s_L is processed just once by the SL policy network p_σ. The output probabilities are stored as prior probabilities P for each legal action a, $P(s, a) = p_\sigma(a|s)$. The leaf node is evaluated in two very different ways: first, by the value network $v_\theta(s_L)$; and second, by the outcome z_L of a random rollout played out until terminal step T using the fast rollout policy p_π; these evaluations are combined, using a mixing parameter λ, into a leaf evaluation $V(s_L)$

$$V(s_L) = (1 - \lambda)v_\theta(s_L) + \lambda z_L$$

At the end of simulation, the action values and visit counts of all traversed edges are updated. Each edge accumulates the visit count and mean evaluation of all simulations passing through that edge

$$N(s, a) = \sum_{i=1}^{n} 1(s, a, i)$$

$$Q(s, a) = \frac{1}{N(s, a)} \sum_{i=1}^{n} 1(s, a, i) V(s_L^i)$$

where s_L^i is the leaf node from the ith simulation, and $1(s, a, i)$ indicates whether an edge (s, a) was traversed during the ith simulation. Once the search is complete, the algorithm chooses the most visited move from the root position.

It is worth noting that the SL policy network p_σ performed better in AlphaGo than the stronger RL policy network p_ρ, presumably because humans select a diverse beam of promising moves, whereas RL optimizes for the single best move. However, the value function $v_\theta(s) \approx v^{p_\rho}(s)$ derived from the stronger RL policy network performed

© 2016 Macmillan Publishers Limited. All rights reserved

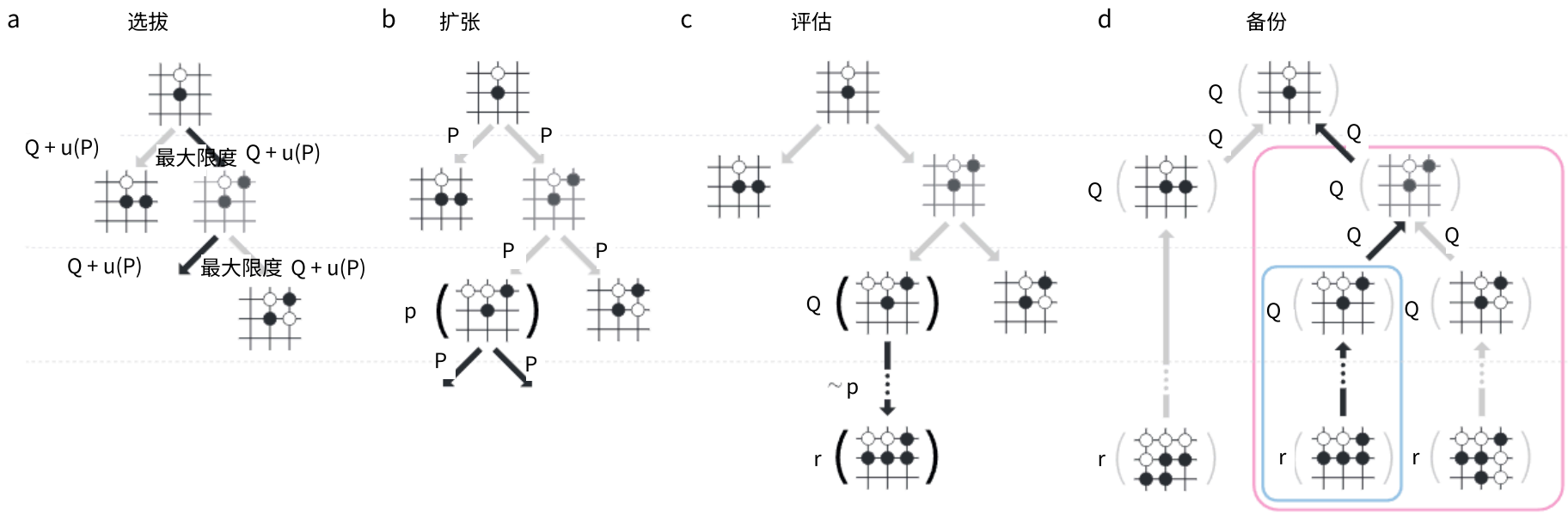


图 3 | AlphaGo 中的蒙特卡洛树搜索。a，每次模拟
a. 遍历决策树，选择动作值 Q 最大的边，并加上一个取决于该边的先验概率 P 的奖励值 u(P)。b. 叶节点可以扩展；新节点由策略网络 p 处理一次，输出概率作为每个动作的先验概率 P 存储。c. 模拟结束时，叶节点评估方式有两种：使用价值网络 v；以及使用快速滚动策略 p 运行滚动到游戏结束，然后使用函数 r 计算获胜者。d，动作值 Q 更新为跟踪该动作下方子树中所有评估 r(·) 和 v(·) 的平均值。

学习卷积神经网络后，在与 Pachi 的比赛中赢得了 11% 的胜利，在与稍弱一些的程序 Fuego 的比赛中赢得了 12% 的胜利。

价值网络的强化学习

训练流程的最后阶段侧重于位置评估，即估计一个价值函数 v(s)，该函数能够根据双方玩家使用策略 p 进行的游戏的位置 s 来预测结果。

$$v^{p}(s) = E[z_t \mid s_t = s, a_{t \cdots T} \sim p]$$

理想情况下，我们希望知道完美博弈下的最优价值函数 v(s)；但在实践中，我们改为估计价值函数。

对于我们最强的策略，我们使用 RL 策略网络 p。我们使用权重为 θ 的价值网络 v(s) 来近似价值函数，

$$v_\theta(s) \approx v^{p}(s) \approx v^{*}(s)$$

该神经网络的架构与策略网络类似，但输出的是单个预测值而非概率分布。我们通过对状态-结果对 (s, z) 进行回归来训练价值网络的权重，并使用随机梯度下降法最小化预测值 v(s) 与相应结果 z 之间的均方误差 (MSE)。

$$\Delta\theta \propto \frac{\partial v_\theta(s)}{\partial \theta}(z - v_\theta(s))$$

直接使用完整对局数据预测对局结果这种简单粗暴的方法会导致过拟合。问题在于，连续的局面高度相关，即使只相差一个棋子，回归目标在整个对局中却是相同的。当使用 KGS 数据集进行训练时，价值网络会记住对局结果，而不是泛化到新的局面，在测试集上的最小均方误差（MSE）为 0.37，而训练集上的 MSE 为 0.19。为了缓解这个问题，我们生成了一个新的自对弈数据集，其中包含 3000 万个不同的局面，每个局面都取自不同的对局。每局对局都是由强化学习策略网络与自身对弈，直到对局结束。在这个数据集上进行训练，训练集和测试集上的 MSE 分别为 0.226 和 0.234，表明过拟合程度很低。图 2b 展示了价值网络的局面评估准确率，并与使用快速滚动策略 p 的蒙特卡罗滚动进行了比较；价值函数始终更加准确。使用 RL 策略网络 p 对 v(s) 进行单次评估，其准确度也接近蒙特卡罗展开，但计算量却减少了 15,000 倍。

利用政策和价值网络进行搜索

AlphaGo 将策略网络和价值网络结合到一个 MCTS 算法中（图 3），该算法通过前瞻搜索来选择行动。

搜索树的每条边 (s, a) 存储着一个动作值 Q(s, a)、访问次数 N(s, a) 和先验概率 P(s, a)。该树通过模拟遍历（即，在不进行备份的情况下，以完整博弈的方式向下遍历该树），从根状态开始。在每次模拟的每个时间步 t，从状态 s 中选择一个动作 a。

$$a_t = \underset{a}{\operatorname{argmax}}(Q(s_t, a) + u(s_t, a))$$

从而最大化行动价值并获得额外奖励

$$u(s, a) \propto \frac{P(s, a)}{1 + N(s, a)}$$

该概率与先验概率成正比，但会随着访问次数的增加而衰减，以鼓励探索。当遍历到达步骤 L 的叶节点时，该叶节点可以扩展。SL 策略网络 p 仅处理叶节点位置一次。输出概率存储为每个合法行为 a 的先验概率 P。

$$P(s, a) = p_\sigma(a \mid s)$$

叶节点的评估方式有两种截然不同的方式：第一种是通过价值网络 v(s)；第二种是通过使用快速滚动策略 p 进行随机滚动直到终止步骤 T 的结果 z；这些评估结果通过混合参数 λ 组合成叶节点评估 V(s)。

$$V(s_L) = (1 - \lambda)v_\theta(s_L) + \lambda z_L$$

模拟结束后，所有经过的边的动作值和访问次数都会更新。每条边都会累加经过该边的所有模拟的访问次数和平均评估值。

$$N(s, a) = \sum_{i=1}^{n} 1(s, a, i)$$

$$Q(s, a) = \frac{1}{N(s, a)} \sum_{i=1}^{n} 1(s, a, i) V(s_L^i)$$

其中 s 是第 i 次模拟中的叶节点，1(s, a, i) 表示在第 i 次模拟中是否遍历了边 (s, a)。搜索完成后，算法会选择从根节点出发访问次数最多的移动。

值得注意的是，SL 策略网络在 AlphaGo 中的表现优于更强大的 RL 策略网络 p，这可能是因为人类会选择一系列多样化的有希望的走法，而 RL 则优化于单一的最佳走法。然而，价值函数

$$v_\theta(s) \approx v^{p_\rho}(s)$$

源自更强的强化学习策略网络

© 2016 麦克米伦出版有限公司。版权所有。

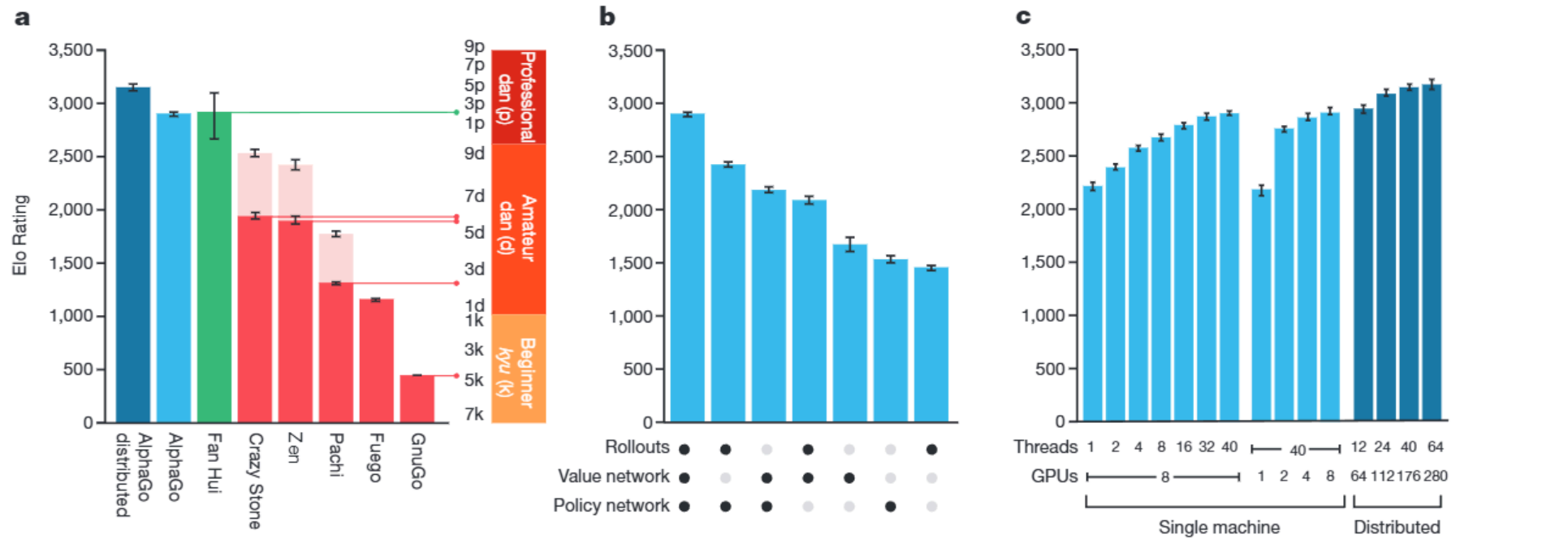


Figure 4 | Tournament evaluation of AlphaGo. a, Results of a tournament between different Go programs (see Extended Data Tables 6–11). Each program used approximately 5 s computation time per move. To provide a greater challenge to AlphaGo, some programs (pale upper bars) were given four handicap stones (that is, free moves at the start of every game) against all opponents. Programs were evaluated on an Elo scale[37]: a 230 point gap corresponds to a 79% probability of winning, which roughly corresponds to one amateur *dan* rank advantage on KGS[38]; an approximate correspondence to human ranks is also shown, horizontal lines show KGS ranks achieved online by that program. Games against the human European champion Fan Hui were also included; these games used longer time controls. 95% confidence intervals are shown. **b**, Performance of AlphaGo, on a single machine, for different combinations of components. The version solely using the policy network does not perform any search. **c**, Scalability study of MCTS in AlphaGo with search threads and GPUs, using asynchronous search (light blue) or distributed search (dark blue), for 2 s per move.

better in AlphaGo than a value function $v_\theta(s) \approx v^{p_\sigma}(s)$ derived from the SL policy network.

Evaluating policy and value networks requires several orders of magnitude more computation than traditional search heuristics. To efficiently combine MCTS with deep neural networks, AlphaGo uses an asynchronous multi-threaded search that executes simulations on CPUs, and computes policy and value networks in parallel on GPUs. The final version of AlphaGo used 40 search threads, 48 CPUs, and 8 GPUs. We also implemented a distributed version of AlphaGo that exploited multiple machines, 40 search threads, 1,202 CPUs and 176 GPUs. The Methods section provides full details of asynchronous and distributed MCTS.

Evaluating the playing strength of AlphaGo

To evaluate AlphaGo, we ran an internal tournament among variants of AlphaGo and several other Go programs, including the strongest commercial programs Crazy Stone[13] and Zen, and the strongest open source programs Pachi[14] and Fuego[15]. All of these programs are based

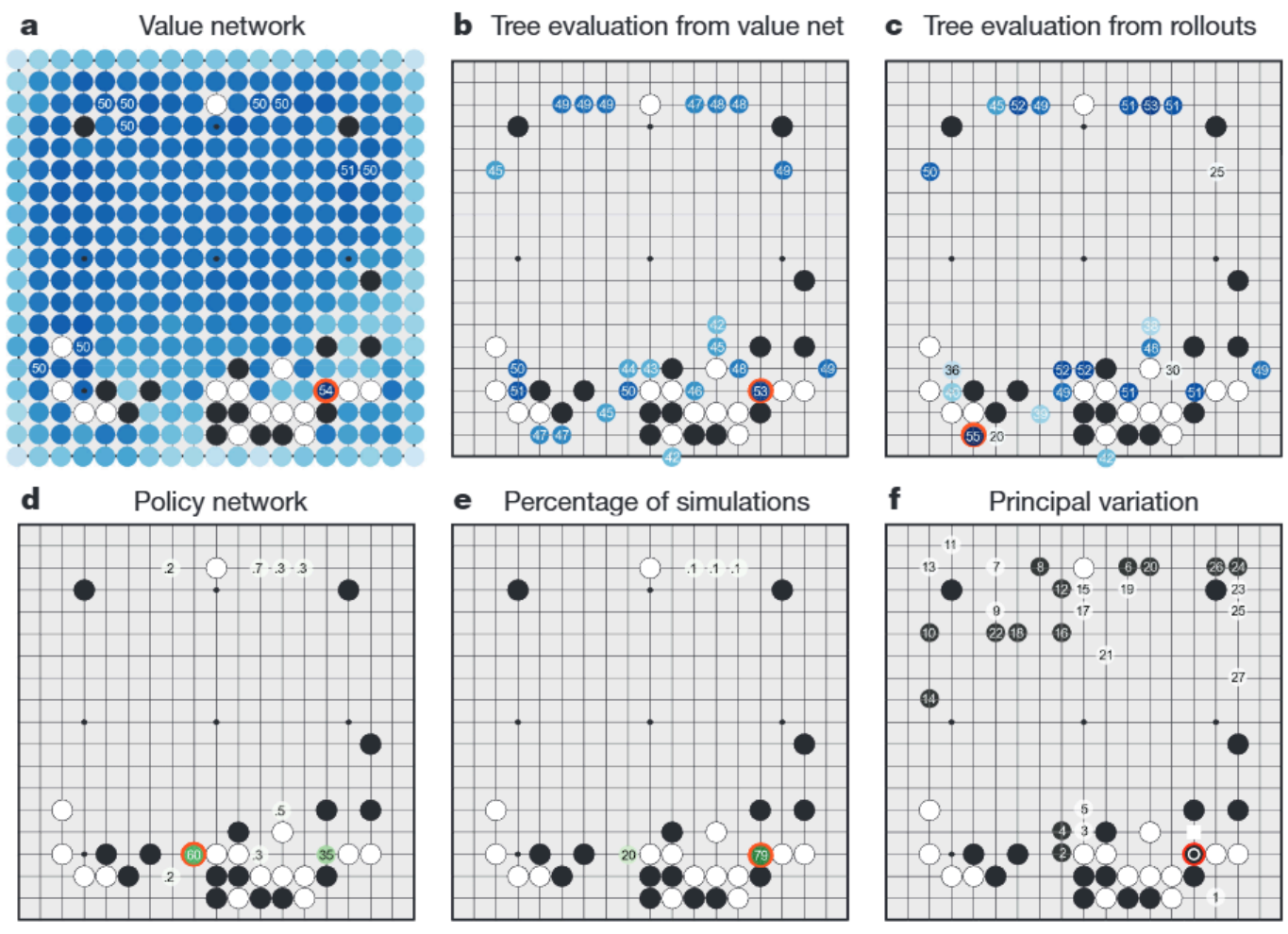


Figure 5 | How AlphaGo (black, to play) selected its move in an informal game against Fan Hui. For each of the following statistics, the location of the maximum value is indicated by an orange circle. **a**, Evaluation of all successors s' of the root position s, using the value network $v_\theta(s')$; estimated winning percentages are shown for the top evaluations. **b**, Action values $Q(s, a)$ for each edge (s, a) in the tree from root position s; averaged over value network evaluations only ($\lambda = 0$). **c**, Action values $Q(s, a)$, averaged over rollout evaluations only ($\lambda = 1$). **d**, Move probabilities directly from the SL policy network, $p_\sigma(a|s)$; reported as a percentage (if above 0.1%). **e**, Percentage frequency with which actions were selected from the root during simulations. **f**, The principal variation (path with maximum visit count) from AlphaGo's search tree. The moves are presented in a numbered sequence. AlphaGo selected the move indicated by the red circle; Fan Hui responded with the move indicated by the white square; in his post-game commentary he preferred the move (labelled 1) predicted by AlphaGo.

© 2016 Macmillan Publishers Limited. All rights reserved

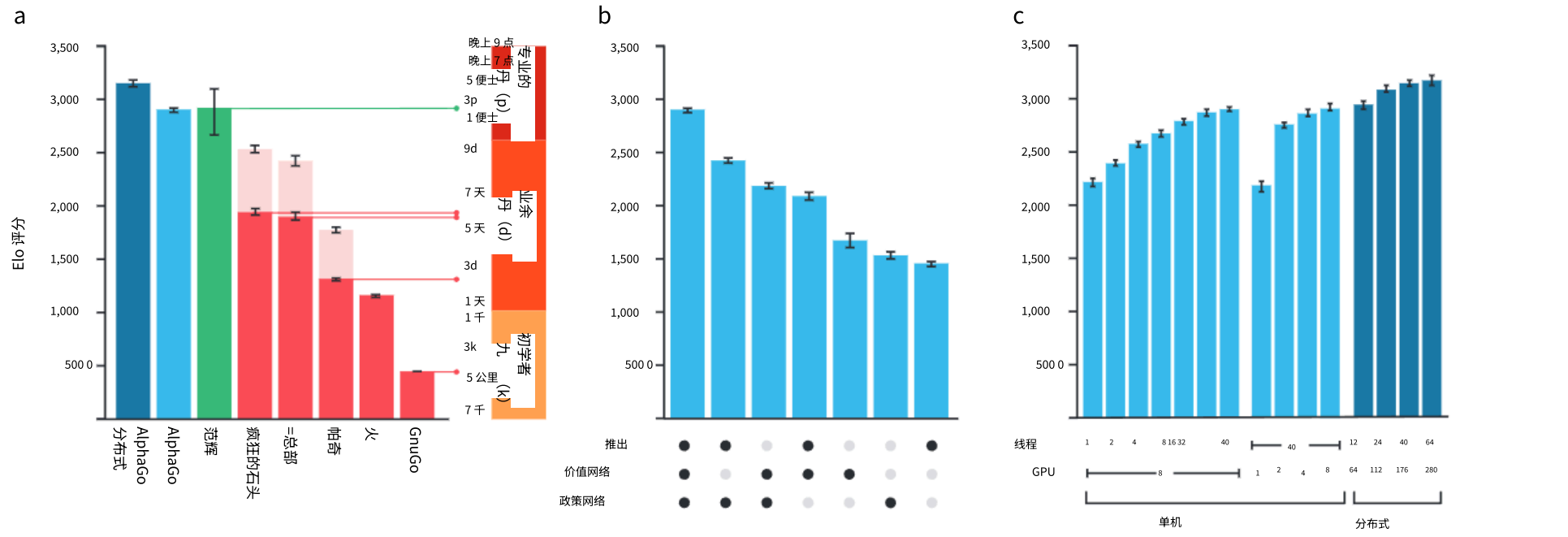


图 4 | AlphaGo 的锦标赛评估。a，比赛结果不同围棋程序之间的比赛（参见扩展数据表 6-11）。每个程序每步棋的计算时间约为 5 秒。为了给 AlphaGo 增加更大的挑战，一些程序（浅色上条）被赋予了四步让子（即每局开始时可以自由走棋），与所有对手对弈。程序评估基于……

Elo 等级分：230 分的差距对应 79%的获胜概率，这大致相当于 KGS 业余棋手段位的优势；图中也显示了与人类等级分的大致对应关系。

在 AlphaGo 中，它比值函数更好。 对比) ≈ᵛ ᴾ% ($ 源自 SL 策略网络。

评估策略网络和价值网络所需的计算量比传统搜索启发式算法高出几个数量级。为了高效地将蒙特卡洛树搜索（MCTS）与深度神经网络相结合，AlphaGo 使用异步多线程搜索，在 CPU 上执行模拟，并在 GPU 上并行计算策略网络和价值网络。最终版本的 AlphaGo 使用了 40 个搜索线程、48 个 CPU 和 8 个 GPU。

我们还实现了 AlphaGo 的分布式版本，该版本利用了多台机器、40 个搜索线程、1202 个 CPU 和 176 个 GPU。“方法”部分提供了异步和分布式 MCTS 的完整细节。

评估 AlphaGo 的对弈实力

为了评估 AlphaGo，我们组织了一场内部比赛，参赛程序包括 AlphaGo 的各种变体以及其他几款围棋程序，其中包括最强的商业程序 Crazy Stone 和 Zen，以及最强的开源程序 Pachi 和 Fuego。所有这些程序都基于……

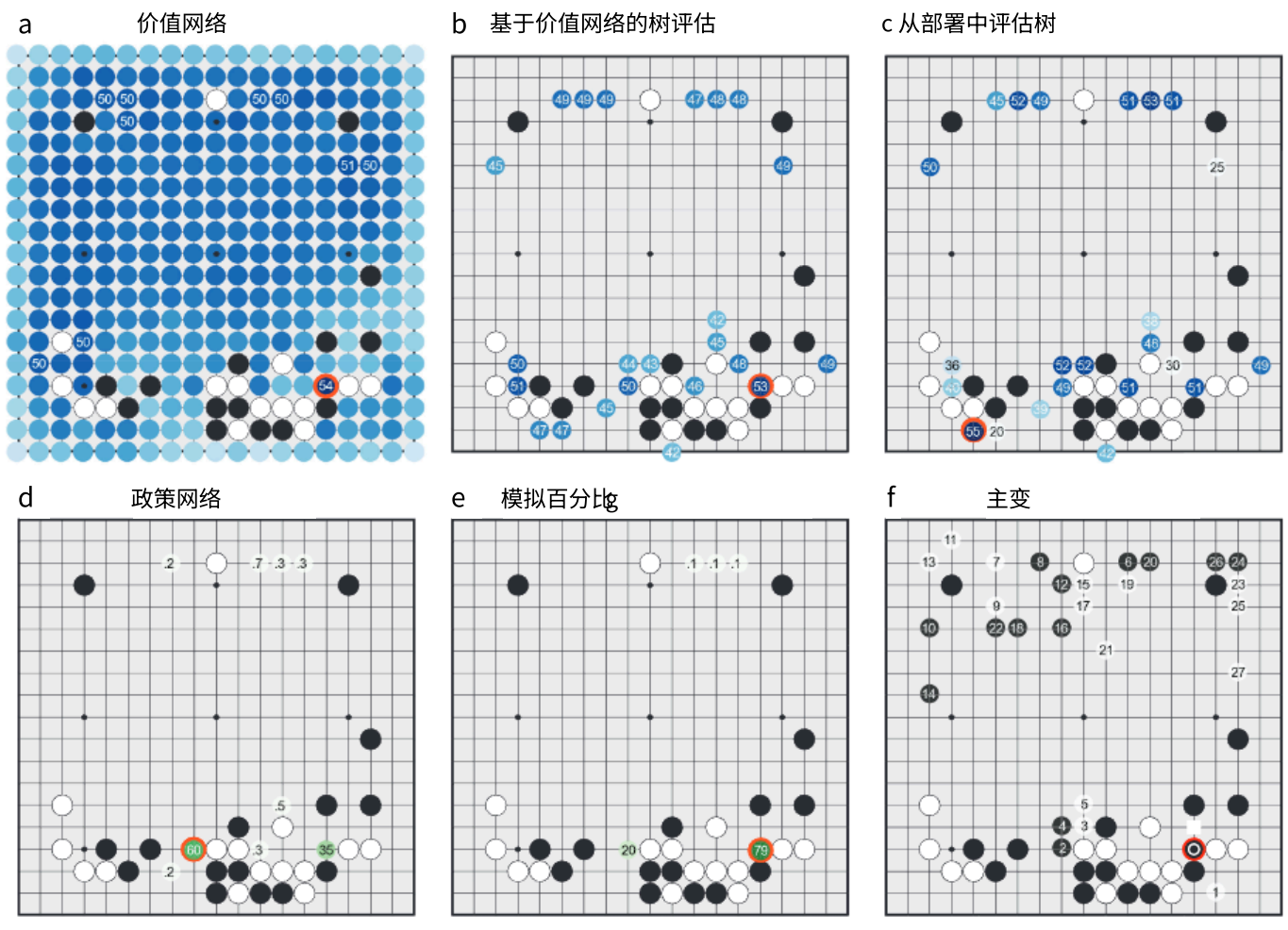


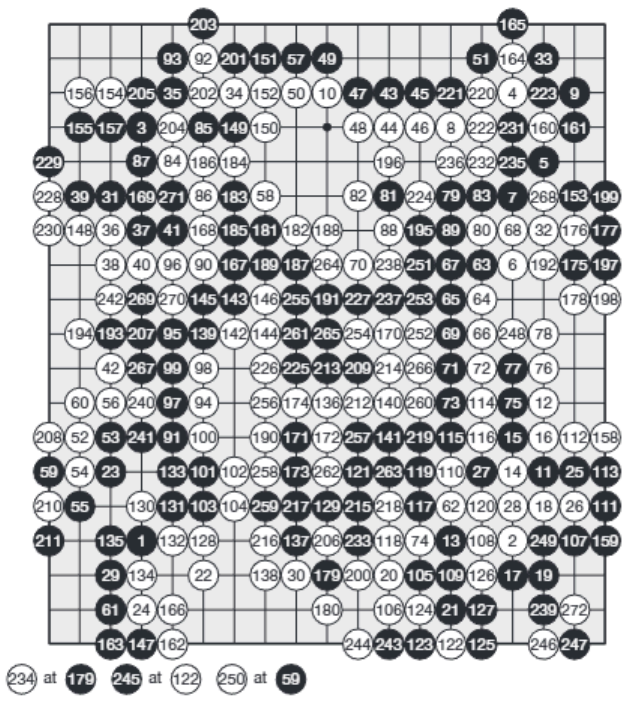
图 5 | AlphaGo（黑棋，轮到你）如何选择它的走法与樊麾的非正式对局。对于以下每项统计数据，最大值的位置用橙色圆圈标出。a，使用价值网络 v(s′) 对根位置 s 的所有后继 s′ 进行评估；图中显示了最高评估的估计胜率。b，从根位置 s 出发的树中每条边 (s, a) 的行动值 Q(s, a)；仅对价值网络评估取平均值 (λ = 0)。

c，行动值 Q(s, a)，仅对推广评估取平均值（λ = 1）。

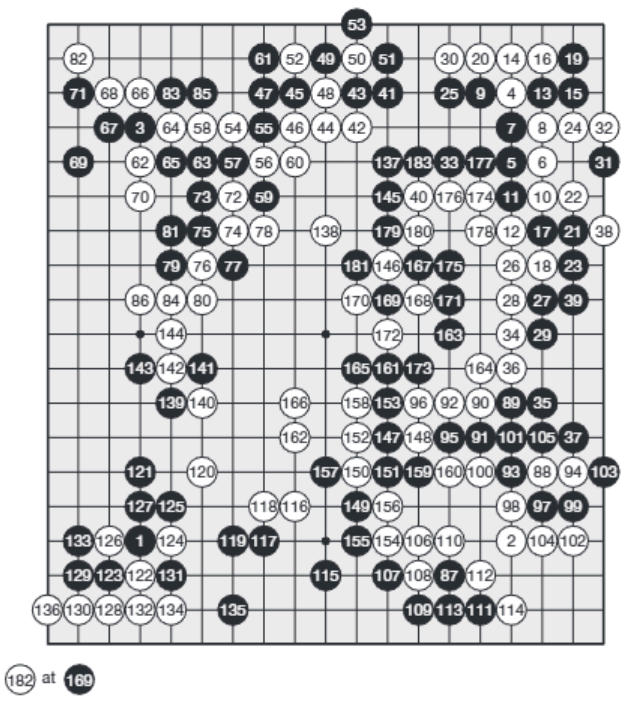
d，直接从 SL 策略网络中转移概率， 帕斯|) 以百分比形式报告（若高于 0.1%）。e，模拟过程中从根节点选择动作的百分比频率。f，AlphaGo 搜索树中的主要变例（访问次数最多的路径）。棋步按编号顺序排列。AlphaGo 选择了红色圆圈所示的棋步；樊麾以白色方块所示的棋步回应；赛后评论中，他更倾向于 AlphaGo 预测的棋步（标记为 1）。

© 2016 麦克米伦出版有限公司。版权所有。

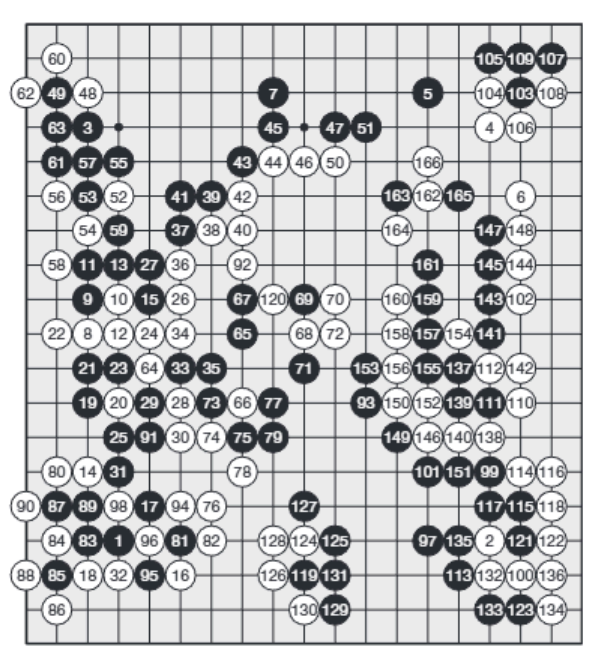
Game 1
Fan Hui (Black), AlphaGo (White)
AlphaGo wins by 2.5 points



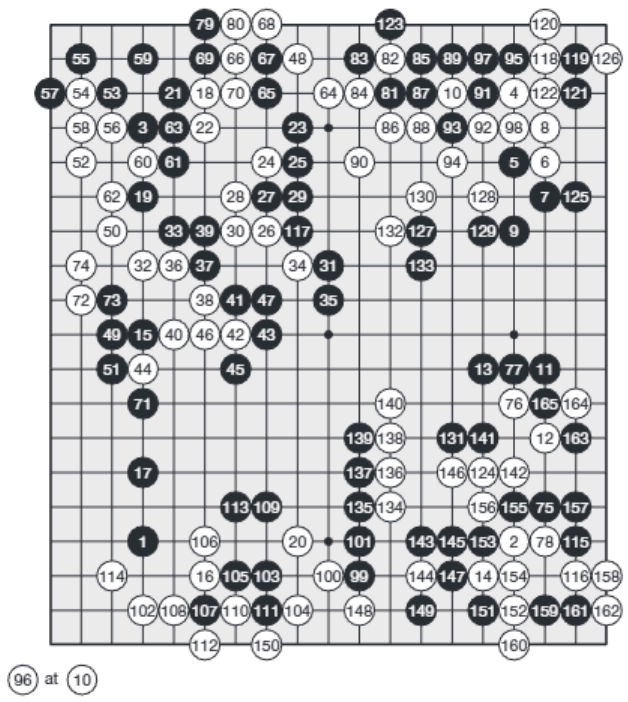
Game 2
AlphaGo (Black), Fan Hui (White)
AlphaGo wins by resignation



Game 3
Fan Hui (Black), AlphaGo (White)
AlphaGo wins by resignation



Game 4
AlphaGo (Black), Fan Hui (White)
AlphaGo wins by resignation



Game 5
Fan Hui (Black), AlphaGo (White)
AlphaGo wins by resignation

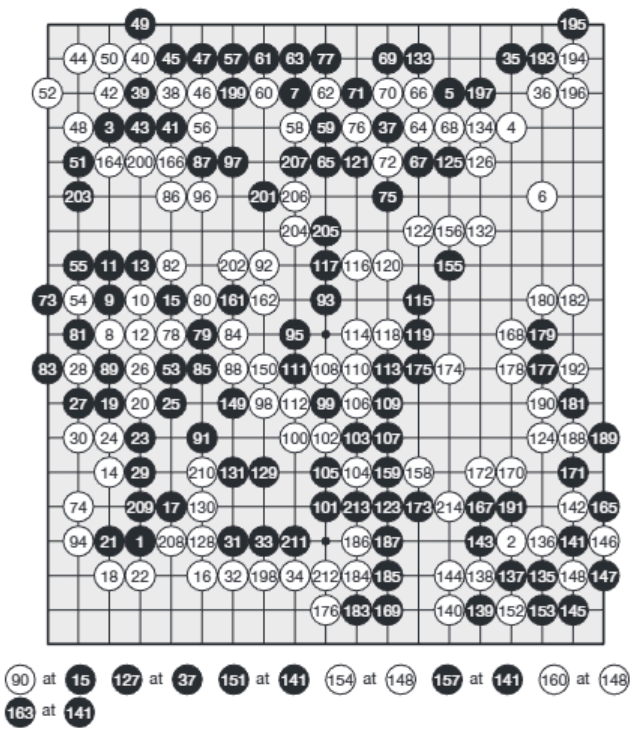


Figure 6 | Games from the match between AlphaGo and the European champion, Fan Hui. Moves are shown in a numbered sequence corresponding to the order in which they were played. Repeated moves on the same intersection are shown in pairs below the board. The first move number in each pair indicates when the repeat move was played, at an intersection identified by the second move number (see Supplementary Information).

on high-performance MCTS algorithms. In addition, we included the open source program GnuGo, a Go program using state-of-the-art search methods that preceded MCTS. All programs were allowed 5 s of computation time per move.

The results of the tournament (see Fig. 4a) suggest that single-machine AlphaGo is many *dan* ranks stronger than any previous Go program, winning 494 out of 495 games (99.8%) against other Go programs. To provide a greater challenge to AlphaGo, we also played games with four handicap stones (that is, free moves for the opponent); AlphaGo won 77%, 86%, and 99% of handicap games against Crazy Stone, Zen and Pachi, respectively. The distributed version of AlphaGo was significantly stronger, winning 77% of games against single-machine AlphaGo and 100% of its games against other programs.

We also assessed variants of AlphaGo that evaluated positions using just the value network ($\lambda = 0$) or just rollouts ($\lambda = 1$) (see Fig. 4b). Even without rollouts AlphaGo exceeded the performance of all other Go programs, demonstrating that value networks provide a viable alternative to Monte Carlo evaluation in Go. However, the mixed evaluation ($\lambda = 0.5$) performed best, winning $\geq$95% of games against other variants. This suggests that the two position-evaluation mechanisms are complementary: the value network approximates the outcome of games played by the strong but impractically slow p_ρ, while the rollouts can precisely score and evaluate the outcome of games played by the weaker but faster rollout policy p_π. Figure 5 visualizes the evaluation of a real game position by AlphaGo.

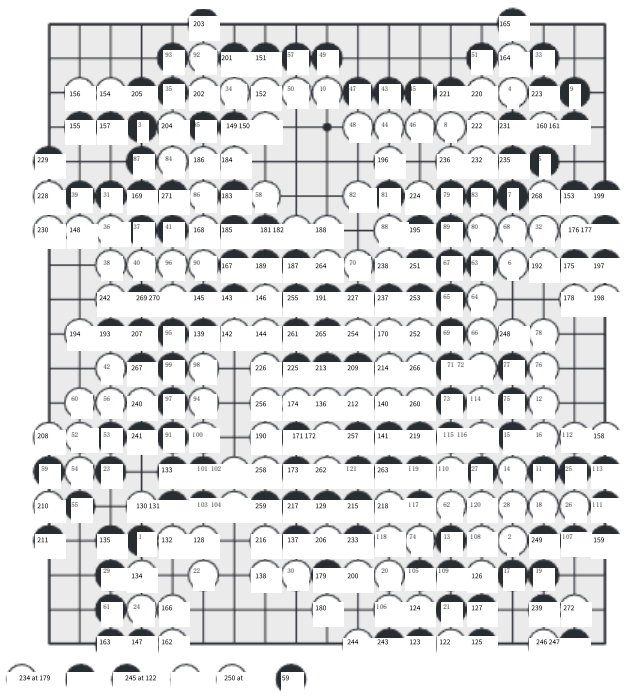
Finally, we evaluated the distributed version of AlphaGo against Fan Hui, a professional 2 *dan*, and the winner of the 2013, 2014 and 2015 European Go championships. Over 5–9 October 2015 AlphaGo and Fan Hui competed in a formal five-game match. AlphaGo won the match 5 games to 0 (Fig. 6 and Extended Data Table 1). This is the first time that a computer Go program has defeated a human professional player, without handicap, in the full game of Go—a feat that was previously believed to be at least a decade away[3,7,31].

Discussion

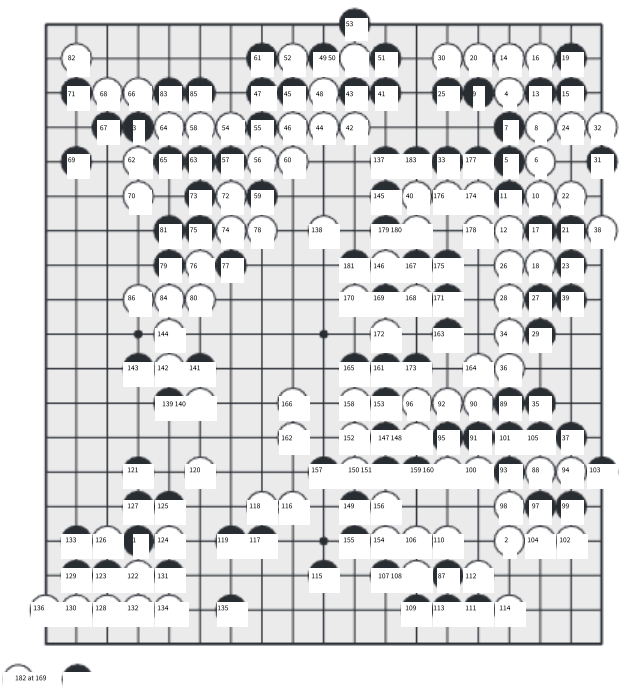
In this work we have developed a Go program, based on a combination of deep neural networks and tree search, that plays at the level of the strongest human players, thereby achieving one of artificial intelligence's "grand challenges"[31–33]. We have developed, for the first time, effective move selection and position evaluation functions for Go, based on deep neural networks that are trained by a novel combination

© 2016 Macmillan Publishers Limited. All rights reserved

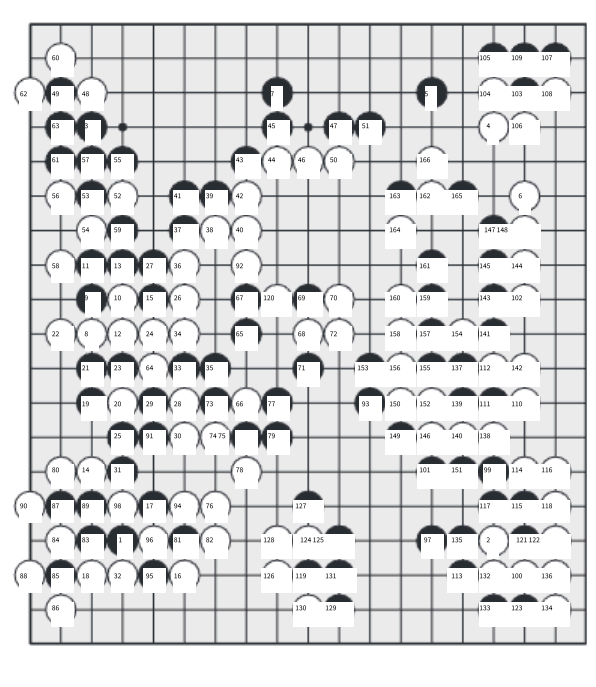
第一局：樊麾（黑），AlphaGo（白），
AlphaGo 胜 2.5 分



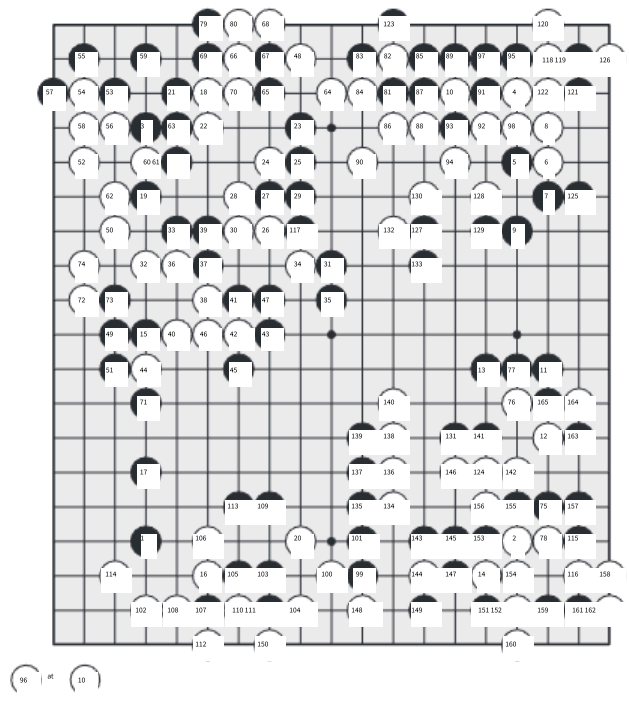
第二局 AlphaGo（黑），樊麾（白）
AlphaGo 因对手认输获胜



第三局 樊麾（黑），AlphaGo（白）
AlphaGo 认输获胜



第四局 AlphaGo（黑），樊麾（白）
AlphaGo 因对手认输获胜



第五局 樊麾（黑），AlphaGo（白）
AlphaGo 认输获胜

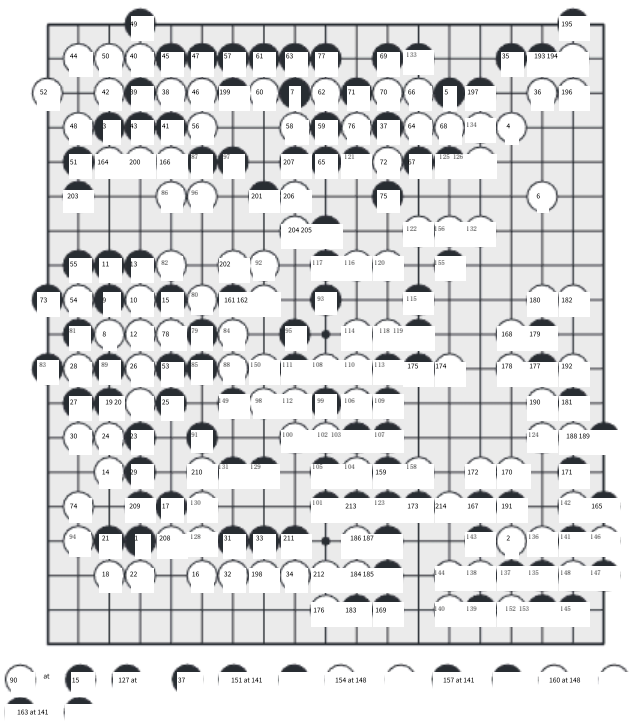


图 6 | AlphaGo 与欧洲队比赛的棋局
冠军樊麾。棋步按顺序编号显示，与实际下棋的顺序一致。同一交叉点上的重复走法成对显示在棋盘下方。第一步每对走法中的走法编号表示重复走法的执行时间，该重复走法位于由第二个走法编号标识的交叉点（参见补充信息）。

我们使用了高性能的蒙特卡洛树搜索（MCTS）算法。此外，我们还加入了开源程序 GnuGo，这是一个使用当时最先进的搜索方法的围棋程序，其搜索方法早于 MCTS。所有程序每步棋的计算时间均限制为 5 秒。

比赛结果（见图 4a）表明，单机版 AlphaGo 的棋艺比以往任何围棋程序都要强很多段，在与其他围棋程序的对局中，495 局赢了 494 局（胜率 99.8%）。为了给 AlphaGo 更大的挑战，我们还进行了四子让子（即对手可以自由走棋）的对局；AlphaGo 在与 Crazy Stone、Zen 和 Pachi 的对局中分别取得了 77%、86% 和 99% 的胜率。分布式版本的 AlphaGo 表现更为出色，在与单机版 AlphaGo 的对局中胜率为 77%，在与其他程序的对局中则保持全胜。

我们还评估了 AlphaGo 的几种变体，它们分别仅使用价值网络（$\lambda = 0$）或仅使用展开（$\lambda = 1$）来评估局面（见图 4b）。即使不使用展开，AlphaGo 的性能也优于所有其他围棋程序，这表明价值网络为围棋中的蒙特卡罗评估提供了一种可行的替代方案。然而，混合评估（$\lambda = 0.5$）表现最佳，在与其他变体的对局中胜率≥95%。

这表明两种局面评估机制是互补的：价值网络近似地反映了实力强劲但速度极慢的策略 p 的对局结果，而展开策略则可以精确地对实力较弱但速度更快的策略 p 的对局结果进行评分和评估。图 5 展示了 AlphaGo 对真实对局局面的评估过程。

最后，我们测试了分布式版本的 AlphaGo 与职业二段棋手樊麾的对弈。樊麾曾获得 2013、2014 和 2015 年欧洲围棋锦标赛冠军。2015 年 10 月 5 日至 9 日，AlphaGo 与樊麾进行了一场正式的五局比赛。AlphaGo 以 5 比 0 的比分获胜（图 6 和扩展数据表 1）。这是计算机围棋程序首次在不让子的情况下，在完整的围棋比赛中击败人类职业棋手——此前人们认为这一壮举至少还需要十年才能实现。

讨论

在这项工作中，我们开发了一个基于深度神经网络和树搜索相结合的围棋程序，其棋艺已达到人类顶尖棋手的水平，从而实现了人工智能领域的一项“重大挑战”。我们首次开发了基于深度神经网络的高效围棋选步和局面评估功能，这些深度神经网络通过一种新颖的组合方法进行训练。

© 2016 麦克米伦出版有限公司。版权所有。

of supervised and reinforcement learning. We have introduced a new search algorithm that successfully combines neural network evaluations with Monte Carlo rollouts. Our program AlphaGo integrates these components together, at scale, in a high-performance tree search engine.

During the match against Fan Hui, AlphaGo evaluated thousands of times fewer positions than Deep Blue did in its chess match against Kasparov[4]; compensating by selecting those positions more intelligently, using the policy network, and evaluating them more precisely, using the value network—an approach that is perhaps closer to how humans play. Furthermore, while Deep Blue relied on a handcrafted evaluation function, the neural networks of AlphaGo are trained directly from gameplay purely through general-purpose supervised and reinforcement learning methods.

Go is exemplary in many ways of the difficulties faced by artificial intelligence[33,34]: a challenging decision-making task, an intractable search space, and an optimal solution so complex it appears infeasible to directly approximate using a policy or value function. The previous major breakthrough in computer Go, the introduction of MCTS, led to corresponding advances in many other domains; for example, general game-playing, classical planning, partially observed planning, scheduling, and constraint satisfaction[35,36]. By combining tree search with policy and value networks, AlphaGo has finally reached a professional level in Go, providing hope that human-level performance can now be achieved in other seemingly intractable artificial intelligence domains.

Online Content Methods, along with any additional Extended Data display items and Source Data, are available in the online version of the paper; references unique to these sections appear only in the online paper.

Received 11 November 2015; accepted 5 January 2016.

1. Allis, L. V. *Searching for Solutions in Games and Artificial Intelligence.* PhD thesis, Univ. Limburg, Maastricht, The Netherlands (1994).
2. van den Herik, H., Uiterwijk, J. W. & van Rijswijck, J. Games solved: now and in the future. *Artif. Intell.* **134,** 277–311 (2002).
3. Schaeffer, J. The games computers (and people) play. *Advances in Computers* **52,** 189–266 (2000).
4. Campbell, M., Hoane, A. & Hsu, F. Deep Blue. *Artif. Intell.* **134,** 57–83 (2002).
5. Schaeffer, J. *et al.* A world championship caliber checkers program. *Artif. Intell.* **53,** 273–289 (1992).
6. Buro, M. From simple features to sophisticated evaluation functions. In *1st International Conference on Computers and Games*, 126–145 (1999).
7. Müller, M. Computer Go. *Artif. Intell.* **134,** 145–179 (2002).
8. Tesauro, G. & Galperin, G. On-line policy improvement using Monte-Carlo search. In *Advances in Neural Information Processing*, 1068–1074 (1996).
9. Sheppard, B. World-championship-caliber Scrabble. *Artif. Intell.* **134,** 241–275 (2002).
10. Bouzy, B. & Helmstetter, B. Monte-Carlo Go developments. In *10th International Conference on Advances in Computer Games*, 159–174 (2003).
11. Coulom, R. Efficient selectivity and backup operators in Monte-Carlo tree search. In *5th International Conference on Computers and Games*, 72–83 (2006).
12. Kocsis, L. & Szepesvári, C. Bandit based Monte-Carlo planning. In *15th European Conference on Machine Learning*, 282–293 (2006).
13. Coulom, R. Computing Elo ratings of move patterns in the game of Go. *ICGA J.* **30,** 198–208 (2007).
14. Baudiš, P. & Gailly, J.-L. Pachi: State of the art open source Go program. In *Advances in Computer Games*, 24–38 (Springer, 2012).
15. Müller, M., Enzenberger, M., Arneson, B. & Segal, R. Fuego – an open-source framework for board games and Go engine based on Monte-Carlo tree search. *IEEE Trans. Comput. Intell. AI in Games* **2,** 259–270 (2010).
16. Gelly, S. & Silver, D. Combining online and offline learning in UCT. In *17th International Conference on Machine Learning*, 273–280 (2007).
17. Krizhevsky, A., Sutskever, I. & Hinton, G. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, 1097–1105 (2012).
18. Lawrence, S., Giles, C. L., Tsoi, A. C. & Back, A. D. Face recognition: a convolutional neural-network approach. *IEEE Trans. Neural Netw.* **8,** 98–113 (1997).
19. Mnih, V. *et al.* Human-level control through deep reinforcement learning. *Nature* **518,** 529–533 (2015).
20. LeCun, Y., Bengio, Y. & Hinton, G. Deep learning. *Nature* **521,** 436–444 (2015).
21. Stern, D., Herbrich, R. & Graepel, T. Bayesian pattern ranking for move prediction in the game of Go. In *International Conference of Machine Learning*, 873–880 (2006).
22. Sutskever, I. & Nair, V. Mimicking Go experts with convolutional neural networks. In *International Conference on Artificial Neural Networks*, 101–110 (2008).
23. Maddison, C. J., Huang, A., Sutskever, I. & Silver, D. Move evaluation in Go using deep convolutional neural networks. *3rd International Conference on Learning Representations* (2015).
24. Clark, C. & Storkey, A. J. Training deep convolutional neural networks to play go. In *32nd International Conference on Machine Learning*, 1766–1774 (2015).
25. Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.* **8,** 229–256 (1992).
26. Sutton, R., McAllester, D., Singh, S. & Mansour, Y. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, 1057–1063 (2000).
27. Sutton, R. & Barto, A. *Reinforcement Learning: an Introduction* (MIT Press, 1998).
28. Schraudolph, N. N., Dayan, P. & Sejnowski, T. J. Temporal difference learning of position evaluation in the game of Go. *Adv. Neural Inf. Process. Syst.* **6,** 817–824 (1994).
29. Enzenberger, M. Evaluation in Go by a neural network using soft segmentation. In *10th Advances in Computer Games Conference*, 97–108 (2003). 267.
30. Silver, D., Sutton, R. & Müller, M. Temporal-difference search in computer Go. *Mach. Learn.* **87,** 183–219 (2012).
31. Levinovitz, A. The mystery of Go, the ancient game that computers still can't win. *Wired Magazine* (2014).
32. Mechner, D. All Systems Go. *The Sciences* **38,** 32–37 (1998).
33. Mandziuk, J. Computational intelligence in mind games. In *Challenges for Computational Intelligence*, 407–442 (2007).
34. Berliner, H. A chronology of computer chess and its literature. *Artif. Intell.* **10,** 201–214 (1978).
35. Browne, C. *et al.* A survey of Monte-Carlo tree search methods. *IEEE Trans. Comput. Intell. AI in Games* **4,** 1–43 (2012).
36. Gelly, S. *et al.* The grand challenge of computer Go: Monte Carlo tree search and extensions. *Commun. ACM* **55,** 106–113 (2012).
37. Coulom, R. Whole-history rating: A Bayesian rating system for players of time-varying strength. In *International Conference on Computers and Games*, 113–124 (2008).
38. KGS. Rating system math. http://www.gokgs.com/help/rmath.html.

Supplementary Information is available in the online version of the paper.

Acknowledgements We thank Fan Hui for agreeing to play against AlphaGo; T. Manning for refereeing the match; R. Munos and T. Schaul for helpful discussions and advice; A. Cain and M. Cant for work on the visuals; P. Dayan, G. Wayne, D. Kumaran, D. Purves, H. van Hasselt, A. Barreto and G. Ostrovski for reviewing the paper; and the rest of the DeepMind team for their support, ideas and encouragement.

Author Contributions A.H., G.v.d.D., J.S., I.A., M.La., A.G., T.G. and D.S. designed and implemented the search in AlphaGo. C.J.M., A.G., L.S., A.H., I.A., V.P., S.D., D.G., N.K., I.S., K.K. and D.S. designed and trained the neural networks in AlphaGo. J.S., J.N., A.H. and D.S. designed and implemented the evaluation framework for AlphaGo. D.S., M.Le., T.L., T.G., K.K. and D.H. managed and advised on the project. D.S., T.G., A.G. and D.H. wrote the paper.

Author Information Reprints and permissions information is available at www.nature.com/reprints. The authors declare no competing financial interests. Readers are welcome to comment on the online version of the paper. Correspondence and requests for materials should be addressed to D.S. (davidsilver@google.com) or D.H. (demishassabis@google.com).

© 2016 Macmillan Publishers Limited. All rights reserved

我们结合了监督学习和强化学习，提出了一种新的搜索算法，该算法成功地将神经网络评估与蒙特卡罗模拟相结合。我们的程序 AlphaGo 将这些组件大规模地集成到一个高性能的树搜索引擎中。在与樊麾的对弈中，AlphaGo 评估的局面数量比“深蓝”在与卡斯帕罗夫的对弈中少了数千倍；它通过策略网络更智能地选择局面，并通过价值网络更精确地评 估局面来弥补这一不足——这种方法或许更接近人类的下棋方式。此外，“深蓝”依赖于手工设计的评估函数，而 AlphaGo 的神经网络则完全通过通用监督学习和强化学习方法，直接从对弈数据中训练而来。

围棋在很多方面都体现了人工智能面临的挑战：决策难度极高，搜索空间庞大且难以处理，最优解极其复杂，以至于似乎无法直接用策略或价值函数进行近似求解。此前计算机围棋领域的一项重大突破——蒙特卡洛树搜索（MCTS）的引入——带动了其他诸多领域的相应进步，例如通用博弈、经典规划、部分可观测规划、调度以及约束满足等。通过将树搜索与策略网络和价值网络相结合，AlphaGo 最终在围棋领域达到了专业水平，这为在其他看似棘手的人工智能领域实现人类水平的竞技带来了希望。

在线内容方法以及任何其他扩展数据显示项目和源数据，均可在论文的在线版本中找到；这些部分特有的参考文献仅出现在在线论文中。

2015 年 11 月 11 日收到；2016 年 1 月 5 日接受。

1. Allis, LV.在游戏和人工智能中寻找解决方案。博士论文，荷兰马斯特里赫特林堡大学（1994）。
2. van den Herik, H.、Uiterwijk, JW 和 van Rijswijck, J. 游戏已解决：现在和未来。阿蒂夫。英特尔。 134, 277–311 (2002)。
3. Schaeffer, J. 计算机（和人）玩的游戏。计算机进展 52, 189–266 (2000)。
4. Campbell, M.、Hoane, A. 和 Hsu, F. 深蓝。阿蒂夫。英特尔。 134, 57–83 (2002)。
5. Schaeffer, J. 等人。世界冠军级跳棋程序。人工智能。53, 273–289 (1992)。
6. Buro, M. 从简单的特征到复杂的评估函数。在第一届计算机与游戏国际会议上，126-145（1999）。
7. Müller, M. 计算机围棋。阿蒂夫。英特尔。 134, 145–179 (2002)。
8. Tesauro, G. 和 Galperin, G. 使用蒙特卡洛搜索进行在线策略改进。在神经信息处理进展中，1068–1074 (1996)。
9. Sheppard, B. 世界冠军级拼字游戏。人工智能 134, 241–275 (2002)。
10. Bouzy, B. & Helmstetter, B. Monte-Carlo Go 发展。在第十届计算机游戏进展国际会议上，159–174 (2003)。
11. Coulom, R. 蒙特卡洛树中的高效选择性和备份算子搜索。在第五届国际计算机与游戏会议上，72-83（2006）。
12. Kocsis, L. 和 Szepesvári, C. 基于 Bandit 的蒙特卡洛规划。在第 15 届欧洲机器学习会议上，282–293 (2006)。
13. Coulom, R. 计算围棋中走法模式的 Elo 等级分。ICGA J. 30, 198–208 (2007)。
14. Baudis, P. & Gailly, J.-L. Pachi: 最先进的开源 Go 程序。载于《计算机游戏进展》，第 24-38 页（Springer，2012 年）。
15. Müller, M.、Enzenberger, M.、Arneson, B. 和 Segal, R. Fuego – 一个基于蒙特卡洛树搜索的棋盘游戏和围棋引擎的开源框架。IEEE 传输。计算。英特尔。游戏中的人工智能 2, 259–270 (2010)。
16. Gelly, S. & Silver, D. 在开普敦大学结合线上和线下学习。在第 17 届国际机器学习大会上，273–280 (2007)。
17. Krizhevsky, A.、Sutskever, I. 和 Hinton, G. 使用深度算法对 ImageNet 进行分类卷积神经网络。神经信息处理系统进展，1097–1105 (2012)。
18. Lawrence, S., Giles, CL, Tsoi, AC & Back, AD 人脸识别：一种卷积神经网络方法。IEEE Trans. Neural Netw. 8, 98–113 (1997)。
19. Mnih, V. 等人。通过深度强化学习实现人类水平的控制。Nature 518, 529–533 (2015)。
20. LeCun, Y., Bengio, Y. & Hinton, G.深度学习。自然 521, 436–444 (2015)。
21. Stern, D.、Herbrich, R. 和 Graepel, T. 移动的贝叶斯模式排序围棋游戏中的预测。国际机器学习会议，873–880 (2006)。
22. Sutskever, I. 和 Nair, V. 利用卷积神经网络模仿围棋专家网络。在国际人工神经网络会议上，101-110（2008）。
23. Maddison, CJ, Huang, A., Sutskever, I. & Silver, D. 使用围棋进行走法评估深度卷积神经网络。第三届国际学习表征会议（2015）。
24. Clark, C. 和 Storkey, AJ 训练深度卷积神经网络下围棋。在第 32 届国际机器学习大会上，1766–1774 (2015)。
25. Williams, RJ.连接主义强化学习的简单统计梯度跟踪算法。机器学习 8，229-256（1992）。
26. Sutton, R.、McAllester, D.、Singh, S. 和 Mansour, Y. 策略梯度方法用于具有函数逼近的强化学习。在神经信息处理系统进展中，1057–1063 (2000)。
27. Sutton, R. & Barto, A. 强化学习：导论（MIT 出版社，1998 年）。
28. Schraudolph, NN, Dayan, P. & Sejnowski, TJ 围棋游戏中位置评估的时间差分学习。Adv. Neural Inf. Process. Syst. 6, 817–824 (1994)。
29. Enzenberger, M. 使用软分割的神经网络对围棋进行评估。第十届计算机游戏进展会议论文集，97-108 (2003)。267。
30. Silver, D.、Sutton, R. 和 Müller, M. 计算机围棋中的时间差分搜索。机器学习 87, 183–219 (2012)。
31. Levinovitz, A.围棋之谜：计算机至今仍无法战胜的古老游戏。《连线》杂志（2014 年）。
32. Mechner, D. 所有系统都运行。科学 38, 32–37 (1998)。
33. Mandziuk, J. 脑力游戏中的计算智能。在《计算智能的挑战》中，407–442 (2007)。
34. Berliner, H. 计算机象棋及其文献年表。人工智能 10, 201–214 (1978)。
35. Browne, C. 等人。蒙特卡洛树搜索方法综述。IEEE Trans. 计算。英特尔。人工智能在游戏中的应用 4, 1–43 (2012)。
36. Gelly, S. 等人。计算机围棋的巨大挑战：蒙特卡洛树搜索和扩展。Commun. ACM 55, 106–113 (2012)。
37. 库洛姆，R. 全历史评分：一种用于玩家的贝叶斯评分系统时变强度。在国际计算机与游戏会议上，113–124 (2008)。
38. KGS。评级系统数学。http://www.gokgs.com/help/rmath.html。

补充信息可在论文的在线版本中找到。

致谢 我们感谢樊麾同意与 AlphaGo 对弈；感谢 T. Manning 担任比赛裁判；感谢 R. Munos 和 T. Schaul 提供的有益讨论和建议；感谢 A. Cain 和 M. Cant 对视觉效果的贡献；感谢 P. Dayan、G. Wayne、D. Kumaran、D. Purves、H. van Hasselt、A. Barreto 和 G. Ostrovski 审阅本文；以及 DeepMind 团队的其他成员的支持、想法和鼓励。

作者贡献：AH、GvdD、JS、IA、M.La.、AG、TG 和 DS 设计并实现了 AlphaGo 中的搜索功能。CJM、AG、LS、AH、IA、VP、SD、DG、NK、IS、KK 和 DS 设计并训练了 AlphaGo 中的神经网络。JS、JN、AH 和 DS 设计并实现了 AlphaGo 的评估框架。DS、M.Le.、TL、TG、KK 和 DH 负责项目管理和咨询。DS、TG、AG 和 DH 撰写了本文。

作者信息、转载和授权信息请访问 www.nature.com/reprints。作者声明无利益冲突。欢迎读者对本文的在线版本发表评论。通讯及材料索取请联系 DS (davidsilver@google.com) 或 DH (demishassabis@google.com)。

© 2016 麦克米伦出版有限公司。版权所有。

METHODS

Problem setting. Many games of perfect information, such as chess, checkers, othello, backgammon and Go, may be defined as alternating Markov games[39]. In these games, there is a state space $\mathcal{S}$ (where state includes an indication of the current player to play); an action space $\mathcal{A}(s)$ defining the legal actions in any given state $s \in \mathcal{S}$; a state transition function $f(s, a, \xi)$ defining the successor state after selecting action a in state s and random input ξ (for example, dice); and finally a reward function $r^i(s)$ describing the reward received by player i in state s. We restrict our attention to two-player zero-sum games, $r^1(s) = -r^2(s) = r(s)$, with deterministic state transitions, $f(s, a, \xi) = f(s, a)$, and zero rewards except at a terminal time step T. The outcome of the game $z_t = \pm r(s_T)$ is the terminal reward at the end of the game from the perspective of the current player at time step t. A policy $p(a|s)$ is a probability distribution over legal actions $a \in \mathcal{A}(s)$. A value function is the expected outcome if all actions for both players are selected according to policy p, that is, $v^p(s) = \mathbb{E}[z_t | s_t = s, a_{t \ldots T} \sim p]$. Zero-sum games have a unique optimal value function $v^*(s)$ that determines the outcome from state s following perfect play by both players,

$$v^*(s) = \begin{cases} z_T & \text{if } s = s_T, \\ \max_a -v^*(f(s,a)) & \text{otherwise} \end{cases}$$

Prior work. The optimal value function can be computed recursively by minimax (or equivalently negamax) search[40]. Most games are too large for exhaustive minimax tree search; instead, the game is truncated by using an approximate value function $v(s) \approx v^*(s)$ in place of terminal rewards. Depth-first minimax search with alpha–beta pruning[40] has achieved superhuman performance in chess[4], checkers[5] and othello[6], but it has not been effective in Go[7].

Reinforcement learning can learn to approximate the optimal value function directly from games of self-play[39]. The majority of prior work has focused on a linear combination $v_\theta(s) = \varphi(s) \cdot \theta$ of features $\varphi(s)$ with weights θ. Weights were trained using temporal-difference learning[41] in chess[42,43], checkers[44,45] and Go[30]; or using linear regression in othello[6] and Scrabble[9]. Temporal-difference learning has also been used to train a neural network to approximate the optimal value function, achieving superhuman performance in backgammon[46]; and achieving weak *kyu*-level performance in small-board Go[28,29,47] using convolutional networks.

An alternative approach to minimax search is Monte Carlo tree search (MCTS)[11,12], which estimates the optimal value of interior nodes by a double approximation, $V^n(s) \approx v^{p^n}(s) \approx v^*(s)$. The first approximation, $V^n(s) \approx v^{p^n}(s)$, uses n Monte Carlo simulations to estimate the value function of a simulation policy P^n. The second approximation, $v^{p^n}(s) \approx v^*(s)$, uses a simulation policy P^n in place of minimax optimal actions. The simulation policy selects actions according to a search control function $\mathbf{argmax}_a \ (Q^n(s,a) + u(s,a))$, such as UCT[12], that selects children with higher action values, $Q^n(s, a) = -V^n(f(s, a))$, plus a bonus $u(s, a)$ that encourages exploration; or in the absence of a search tree at state s, it samples actions from a fast rollout policy $p_\pi(a|s)$. As more simulations are executed and the search tree grows deeper, the simulation policy becomes informed by increasingly accurate statistics. In the limit, both approximations become exact and MCTS (for example, with UCT) converges[12] to the optimal value function $\lim_{n\to\infty} V^n(s) = \lim_{n\to\infty} v^{p^n}(s) = v^*(s)$. The strongest current Go programs are based on MCTS[13–15,36].

MCTS has previously been combined with a policy that is used to narrow the beam of the search tree to high-probability moves[13]; or to bias the bonus term towards high-probability moves[48]. MCTS has also been combined with a value function that is used to initialize action values in newly expanded nodes[16], or to mix Monte Carlo evaluation with minimax evaluation[49]. By contrast, AlphaGo's use of value functions is based on truncated Monte Carlo search algorithms[8,9], which terminate rollouts before the end of the game and use a value function in place of the terminal reward. AlphaGo's position evaluation mixes full rollouts with truncated rollouts, resembling in some respects the well-known temporal-difference learning algorithm TD(λ). AlphaGo also differs from prior work by using slower but more powerful representations of the policy and value function; evaluating deep neural networks is several orders of magnitude slower than linear representations and must therefore occur asynchronously.

The performance of MCTS is to a large degree determined by the quality of the rollout policy. Prior work has focused on handcrafted patterns[50] or learning rollout policies by supervised learning[13], reinforcement learning[16], simulation balancing[51,52] or online adaptation[30,53]; however, it is known that rollout-based position evaluation is frequently inaccurate[54]. AlphaGo uses relatively simple rollouts, and instead addresses the challenging problem of position evaluation more directly using value networks.

Search algorithm. To efficiently integrate large neural networks into AlphaGo, we implemented an asynchronous policy and value MCTS algorithm (APV-MCTS). Each node s in the search tree contains edges (s, a) for all legal actions $a \in \mathcal{A}(s)$. Each edge stores a set of statistics,

$$\{P(s,a),\ N_v(s,a),\ N_r(s,a),\ W_v(s,a),\ W_r(s,a),\ Q(s,a)\}$$

where $P(s, a)$ is the prior probability, $W_v(s, a)$ and $W_r(s, a)$ are Monte Carlo estimates of total action value, accumulated over $N_v(s, a)$ and $N_r(s, a)$ leaf evaluations and rollout rewards, respectively, and $Q(s, a)$ is the combined mean action value for that edge. Multiple simulations are executed in parallel on separate search threads. The APV-MCTS algorithm proceeds in the four stages outlined in Fig. 3.

Selection (Fig. 3a). The first in-tree phase of each simulation begins at the root of the search tree and finishes when the simulation reaches a leaf node at time step L. At each of these time steps, $t < L$, an action is selected according to the statistics in the search tree, $a_t = \mathbf{argmax}_a \ (Q(s_t, a) + u(s_t, a))$ using a variant of the PUCT algorithm[48], $u(s,a) = c_{\text{puct}} P(s,a) \frac{\sqrt{\sum_b N_r(s,b)}}{1 + N_r(s,a)}$, where c_{puct} is a constant determining the level of exploration; this search control strategy initially prefers actions with high prior probability and low visit count, but asymptotically prefers actions with high action value.

Evaluation (Fig. 3c). The leaf position s_L is added to a queue for evaluation $v_\theta(s_L)$ by the value network, unless it has previously been evaluated. The second rollout phase of each simulation begins at leaf node s_L and continues until the end of the game. At each of these time-steps, $t \geq L$, actions are selected by both players according to the rollout policy, $a_t \sim p_\pi(\cdot|s_t)$. When the game reaches a terminal state, the outcome $z_t = \pm r(s_T)$ is computed from the final score.

Backup (Fig. 3d). At each in-tree step $t \leq L$ of the simulation, the rollout statistics are updated as if it has lost n_{vl} games, $N_r(s_t, a_t) \leftarrow N_r(s_t, a_t) + n_{\text{vl}}$; $W_r(s_t, a_t) \leftarrow W_r(s_t, a_t) - n_{\text{vl}}$; this virtual loss[55] discourages other threads from simultaneously exploring the identical variation. At the end of the simulation, t he rollout statistics are updated in a backward pass through each step $t \leq L$, replacing the virtual losses by the outcome, $N_r(s_t, a_t) \leftarrow N_r(s_t, a_t) - n_{\text{vl}} + 1$; $W_r(s_t, a_t) \leftarrow W_r(s_t, a_t) + n_{\text{vl}} + z_t$. Asynchronously, a separate backward pass is initiated when the evaluation of the leaf position s_L completes. The output of the value network $v_\theta(s_L)$ is used to update value statistics in a second backward pass through each step $t \leq L$, $N_v(s_t, a_t) \leftarrow N_v(s_t, a_t) + 1$, $W_v(s_t, a_t) \leftarrow W_v(s_t, a_t) + v_\theta(s_L)$. The overall evaluation of each state action is a weighted average of the Monte Carlo estimates, $Q(s,a) = (1-\lambda)\frac{W_v(s,a)}{N_v(s,a)} + \lambda \frac{W_r(s,a)}{N_r(s,a)}$, that mixes together the value network and rollout evaluations with weighting parameter λ. All updates are performed lock-free[56].

Expansion (Fig. 3b). When the visit count exceeds a threshold, $N_r(s, a) > n_{\text{thr}}$, the successor state $s' = f(s, a)$ is added to the search tree. The new node is initialized to $\{N(s', a) = N_r(s', a) = 0, W(s', a) = W_r(s', a) = 0, P(s', a) = p_\sigma(a|s')\}$, using a tree policy $p_\tau(a|s')$ (similar to the rollout policy but with more features, see Extended Data Table 4) to provide placeholder prior probabilities for action selection. The position s' is also inserted into a queue for asynchronous GPU evaluation by the policy network. Prior probabilities are computed by the SL policy network $p_\sigma^\beta(\cdot|s')$ with a softmax temperature set to β; these replace the placeholder prior probabilities, $P(s', a) \leftarrow p_\sigma^\beta(a|s')$, using an atomic update. The threshold n_{thr} is adjusted dynamically to ensure that the rate at which positions are added to the policy queue matches the rate at which the GPUs evaluate the policy network. Positions are evaluated by both the policy network and the value network using a mini-batch size of 1 to minimize end-to-end evaluation time.

We also implemented a distributed APV-MCTS algorithm. This architecture consists of a single master machine that executes the main search, many remote worker CPUs that execute asynchronous rollouts, and many remote worker GPUs that execute asynchronous policy and value network evaluations. The entire search tree is stored on the master, which only executes the in-tree phase of each simulation. The leaf positions are communicated to the worker CPUs, which execute the rollout phase of simulation, and to the worker GPUs, which compute network features and evaluate the policy and value networks. The prior probabilities of the policy network are returned to the master, where they replace placeholder prior probabilities at the newly expanded node. The rewards from rollouts and the value network outputs are each returned to the master, and backed up the originating search path.

At the end of search AlphaGo selects the action with maximum visit count; this is less sensitive to outliers than maximizing action value[15]. The search tree is reused at subsequent time steps: the child node corresponding to the played action becomes the new root node; the subtree below this child is retained along with all its statistics, while the remainder of the tree is discarded. The match version of AlphaGo continues searching during the opponent's move. It extends the search

© 2016 Macmillan Publishers Limited. All rights reserved

研究 文章

方法

问题设定。许多完全信息博弈，例如国际象棋、跳棋、奥赛罗棋、西洋双陆棋和围棋，都可以定义为交替马尔可夫博弈。在这些博弈中，存在一个状态空间 S（其中状态包含当前玩家的指示）；一个动作空间（）。

定义任意给定状态 s ∈ S 下的合法行为；状态转移函数 f(s, a, ξ) 定义在状态 s 中选择行为 a 并输入随机数 ξ（例如，骰子）后的后继状态；最后，奖励函数 r(s) 描述玩家 i 在状态 s 下获得的奖励。我们仅关注双人零和博弈，r(s) = −r(s) = r(s)，状态转移为确定性函数 f(s, a, ξ) = f(s, a)，且除终止时间步 T 外，奖励均为零。博弈结果 z = ±r(s) 是当前玩家在时间步 t 的博弈结束时的最终奖励。策略 p(a|s) 是合法行为 a ∈ ( ) 的概率分布。

作为 。

价值函数是指当双方玩家的所有行动都按照策略 p 选择时，预期的结果，即 E。

零和博弈具有唯一的最优值函数 ( )，该函数决定了双方玩家在状态 s 下进行完美博弈的结果。

$$v^*(s) = \begin{cases} z_T & \text{如果 } s = s_T, \\ \max_a v^*(f(s,a)) & \text{最大限度 } (\text{金融服务管理局} \end{cases}$$

前期工作。最优值函数可以通过极小极大（或等价的负极大）搜索递归计算。大多数游戏规模过大，无法进行穷举的极小极大树搜索；因此，通常使用近似值函数 v(s) ≈ v(s) 来代替最终奖励，从而对游戏进行截断。采用 alpha-beta 剪枝的深度优先极小极大搜索在国际象棋、跳棋和奥赛罗棋中取得了超越人类的水平，但在围棋中效果不佳。

强化学习可以直接从自我对弈游戏中学习逼近最优值函数。以往的大部分工作都集中在特征ϕ(s)及其权重θ的线性组合 v(s) = ϕ(s) · θ上。权重的训练方法包括：在国际象棋、跳棋和围棋中使用时序差分学习；在奥赛罗和拼字游戏中使用线性回归。时序差分学习也被用于训练神经网络以逼近最优值函数，并在西洋双陆棋中实现了超人水平；以及使用卷积神经网络在小棋盘围棋中实现了弱级水平。

蒙特卡洛树搜索（MCTS）是极小极大搜索的另一种方法，它通过双重近似来估计内部节点的最优值。

，使用 n 次蒙特卡罗模拟来估计模拟策略 P 的价值函数。第二个近似值，

，使用模拟策略 P 代替极小极大最优动作。模拟策略根据搜索控制函数 (( ) + ( )) 选择动作。

例如 UCT，它选择具有较高动作值的儿童，Q(s, a) = 关闭(s, a))，加上鼓励探索的奖励 u(s, a)；或者在状态 s 处没有搜索树时，它从快速展开策略中采样动作 (|)

随着模拟次数的增加和搜索树的深度增加，模拟策略会越来越依赖于精确的统计数据。最终，两种近似方法都将变得精确，MCTS（例如，使用 UCT）会收敛到最优值函数。

对比 目前最强的 Go 程序是基于蒙特卡洛树搜索法。

蒙特卡洛搜索算法（MCTS）此前曾与一种策略结合使用，该策略用于将搜索树的范围缩小到高概率走法；或者将奖励项偏向高概率走法。MCTS 也曾与一种价值函数结合使用，该价值函数用于初始化新扩展节点中的动作值，或者将蒙特卡洛评估与极小极大评估相结合。相比之下，AlphaGo 对价值函数的使用基于截断蒙特卡洛搜索算法，该算法会在游戏结束前终止迭代，并使用价值函数代替最终奖励。AlphaGo 的位置评估混合了完整的迭代和截断的迭代，在某些方面类似于著名的时序差分学习算法 TD(λ)。AlphaGo 与以往工作的另一个不同之处在于，它使用了速度较慢但功能更强大的策略和价值函数表示；评估深度神经网络比线性表示慢几个数量级，因此必须异步进行。

蒙特卡洛树搜索（MCTS）的性能很大程度上取决于展开策略的质量。以往的研究主要集中于手工设计模式或通过监督学习、强化学习、模拟平衡或在线自适应等方法来学习展开策略；然而，众所周知，基于展开的局面评估通常不够准确。AlphaGo 使用了相对简单的展开策略，而是利用价值网络更直接地解决了局面评估这一难题。

搜索算法。为了高效地将大型神经网络集成到 AlphaGo 中，我们实现了一种异步策略和值蒙特卡洛树搜索算法（APV-MCTS）。搜索树中的每个节点 s 都包含边 (s, a)，表示所有合法动作 a ∈ ( )。 作为 。

每条边都存储着一组统计数据，

$$\{P(s,a),\ N_v(s,a),\ N_r(s,a),\ W_v(s,a),\ W_r(s,a),\ Q(s,a)\}$$

其中 P(s, a) 为先验概率，W(s, a) 和 W(s, a) 分别为基于 N(s, a) 和 N(s, a) 次叶节点评估和展开奖励累积的总动作价值的蒙特卡罗估计值，Q(s, a) 为该边的综合平均动作价值。多个模拟在不同的搜索线程上并行执行。

APV-MCTS 算法按图 3 所示的四个阶段进行。选择阶段（图 3a）。每次模拟的第一个树内阶段从搜索树的根节点开始，并在时间步长 L 到达叶节点时结束。在每个时间步长 t < L 时，根据搜索树中的统计数据选择一个动作，即 = (( ) + ( ))。

$$a_t \quad \text{argmax}_a \quad Q(s,a), \qquad \text{美国}, \qquad \text{使用 PUCT 的变体}$$

算法，cis 是决定探索程度的常数；) 这种搜索控制策略最初倾向于先验概率高、访问次数少的动作，但渐近地倾向于动作价值高的动作。

评估（图 3c）。除非叶节点位置之前已被评估过，否则它将被价值网络添加到评估队列 v(s) 中。每次模拟的第二个展开阶段从叶节点开始，并持续到游戏结束。在每个时间步 t ⩾ L 中，双方玩家根据展开策略 ~ (·|) 选择行动。

$$a_t \quad p_\pi \quad \text{st. 当游戏达到终止状态时，}$$

结果 $z_t = \pm r(s_T)$ 由最终得分计算得出。

备份（图 3d）。在模拟的每个树内步骤 t ⩽ L 中，展开统计信息会更新，如同它输掉了 n 局游戏：N(s, a) ← N(s, a) + n；W(s, a) ← W(s, a) −n；这种虚拟损失会阻止其他线程同时探索相同的变体。在模拟结束时，展开统计信息会通过反向传播更新，遍历每个步骤 t ⩽ L，并将虚拟损失替换为结果：N(s, a) ← N(s, a) −n+1；W(s, a) ← W(s, a) + n+z。异步地，当叶节点位置的评估完成后，会启动一个单独的反向传播。价值网络 v(s) 的输出用于在第二次反向传播中更新价值统计信息，其中每个步骤 t ⩽ L，N(s, a) ← N(s, a) + 1，W(s, a) ← W(s, a) + v(s)。每个状态动作的总体评估是蒙特卡罗估计的加权平均值。

这种方法将价值网络和展开评估结果混合在一起，并使用权重参数 λ 进行加权。所有更新均以无锁方式执行。扩展（图 3b）。当访问次数超过阈值 N(s, a) > n 时，后继状态 s′ = f(s, a) 被添加到搜索树中。新节点初始化为 {N(s′, a) = N(s′, a) = 0, W(s′, a) = W(s′, a) = 0, P(s′,a) = p(a|s′)}，其中使用树策略 p(a|s′)（类似于展开策略，但具有更多特征，参见扩展数据表 4）为动作选择提供占位符先验概率。位置 s′ 也被插入到队列中，以便策略网络进行异步 GPU 评估。先验概率由 SL 策略网络计算。

$$p_\sigma^\beta(s')$$

softmax 温度设置为 β；这些替换了占位符先验概率，(′) ← (| ′)

采用原子更新。阈值会动态调整，以确保位置添加到策略队列的速率与 GPU 评估策略网络的速率相匹配。位置由策略网络和值网络共同评估，小批量大小为 1，以最大限度地缩短端到端评估时间。

我们还实现了一种分布式 APV-MCTS 算法。该架构由一台主节点（执行主搜索）、多个远程工作节点（执行异步展开）和多个远程工作节点（执行异步策略网络和价值网络评估）组成。整个搜索树存储在主节点上，主节点仅执行每次模拟的树内阶段。叶节点的位置信息被传递给执行模拟展开阶段的工作节点和计算网络特征并评估策略网络和价值网络的工作节点。策略网络的先验概率被返回给主节点，并在新扩展的节点上替换占位符先验概率。展开的奖励和价值网络的输出分别返回给主节点，并备份到原始搜索路径。

搜索结束后，AlphaGo 会选择访问次数最多的动作；与最大化动作价值相比，这种方法对异常值的敏感度较低。搜索树会在后续时间步中重复使用：与已执行动作对应的子节点会成为新的根节点；该子节点下的子树及其所有统计数据会被保留，而树的其余部分则会被丢弃。比赛版本的 AlphaGo 会在对手走棋期间继续搜索，从而扩展搜索范围。

© 2016 麦克米伦出版有限公司。版权所有。

if the action maximizing visit count and the action maximizing action value disagree. Time controls were otherwise shaped to use most time in the middle-game[57]. AlphaGo resigns when its overall evaluation drops below an estimated 10% probability of winning the game, that is, $\max_a Q(s,a) < -0.8$.

AlphaGo does not employ the all-moves-as-first[10] or rapid action value estimation[58] heuristics used in the majority of Monte Carlo Go programs; when using policy networks as prior knowledge, these biased heuristics do not appear to give any additional benefit. In addition AlphaGo does not use progressive widening[13], dynamic *komi*[59] or an opening book[60]. The parameters used by AlphaGo in the Fan Hui match are listed in Extended Data Table 5.

Rollout policy. The rollout policy $p_\pi(a|s)$ is a linear softmax policy based on fast, incrementally computed, local pattern-based features consisting of both 'response' patterns around the previous move that led to state s, and 'non-response' patterns around the candidate move a in state s. Each non-response pattern is a binary feature matching a specific 3×3 pattern centred on a, defined by the colour (black, white, empty) and liberty count (1, 2, $\geq$3) for each adjacent intersection. Each response pattern is a binary feature matching the colour and liberty count in a 12-point diamond-shaped pattern[21] centred around the previous move. Additionally, a small number of handcrafted local features encode common-sense Go rules (see Extended Data Table 4). Similar to the policy network, the weights π of the rollout policy are trained from 8 million positions from human games on the Tygem server to maximize log likelihood by stochastic gradient descent. Rollouts execute at approximately 1,000 simulations per second per CPU thread on an empty board.

Our rollout policy $p_\pi(a|s)$ contains less handcrafted knowledge than state-of-the-art Go programs[13]. Instead, we exploit the higher-quality action selection within MCTS, which is informed both by the search tree and the policy network. We introduce a new technique that caches all moves from the search tree and then plays similar moves during rollouts; a generalization of the 'last good reply' heuristic[53]. At every step of the tree traversal, the most probable action is inserted into a hash table, along with the 3×3 pattern context (colour, liberty and stone counts) around both the previous move and the current move. At each step of the rollout, the pattern context is matched against the hash table; if a match is found then the stored move is played with high probability.

Symmetries. In previous work, the symmetries of Go have been exploited by using rotationally and reflectionally invariant filters in the convolutional layers[24,28,29]. Although this may be effective in small neural networks, it actually hurts performance in larger networks, as it prevents the intermediate filters from identifying specific asymmetric patterns[23]. Instead, we exploit symmetries at run-time by dynamically transforming each position s using the dihedral group of eight reflections and rotations, $d_1(s), \ldots, d_8(s)$. In an explicit symmetry ensemble, a mini-batch of all 8 positions is passed into the policy network or value network and computed in parallel. For the value network, the output values are simply averaged, $\overline{v}_\theta(s) = \frac{1}{8}\sum_{j=1}^{8} v_\theta(d_j(s))$. For the policy network, the planes of output probabilities are rotated/reflected back into the original orientation, and averaged together to provide an ensemble prediction, $\overline{p}_\sigma(\cdot|s) = \frac{1}{8}\sum_{j=1}^{8} d_j^{-1}(p_\sigma(\cdot|d_j(s)))$; this approach was used in our raw network evaluation (see Extended Data Table 3). Instead, APV-MCTS makes use of an implicit symmetry ensemble that randomly selects a single rotation/reflection $j \in [1, 8]$ for each evaluation. We compute exactly one evaluation for that orientation only; in each simulation we compute the value of leaf node s_L by $v_\theta(d_j(s_L))$, and allow the search procedure to average over these evaluations. Similarly, we compute the policy network for a single, randomly selected rotation/reflection, $d_j^{-1}(p_\sigma(\cdot|d_j(s)))$.

Policy network: classification. We trained the policy network p_σ to classify positions according to expert moves played in the KGS data set. This data set contains 29.4 million positions from 160,000 games played by KGS 6 to 9 *dan* human players; 35.4% of the games are handicap games. The data set was split into a test set (the first million positions) and a training set (the remaining 28.4 million positions). Pass moves were excluded from the data set. Each position consisted of a raw board description s and the move a selected by the human. We augmented the data set to include all eight reflections and rotations of each position. Symmetry augmentation and input features were pre-computed for each position. For each training step, we sampled a randomly selected mini-batch of m samples from the augmented KGS data set, $\{s^k, a^k\}_{k=1}^{m}$ and applied an asynchronous stochastic gradient descent update to maximize the log likelihood of the action, $\Delta\sigma = \frac{\alpha}{m}\sum_{k=1}^{m} \frac{\partial \log p_\sigma(a^k|s^k)}{\partial\sigma}$. The step size α was initialized to 0.003 and was halved every 80 million training steps, without momentum terms, and a mini-batch size of $m = 16$. Updates were applied asynchronously on 50 GPUs using DistBelief[61]; gradients older than 100 steps were discarded. Training took around 3 weeks for 340 million training steps.

Policy network: reinforcement learning. We further trained the policy network by policy gradient reinforcement learning[25,26]. Each iteration consisted of a mini-batch of n games played in parallel, between the current policy network p_ρ that is being trained, and an opponent p_{ρ^-} that uses parameters ρ^- from a previous iteration, randomly sampled from a pool of opponents, so as to increase the stability of training. Weights were initialized to $\rho = \rho^- = \sigma$. Every 500 iterations, we added the current parameters ρ to the opponent pool. Each game i in the mini-batch was played out until termination at step T^i, and then scored to determine the outcome $z_t^i = \pm r(s_{T^i})$ from each player's perspective. The games were then replayed to determine the policy gradient update, $\Delta\rho = \frac{\alpha}{n}\sum_{i=1}^{n}\sum_{t=1}^{T^i} \frac{\partial \log p_\rho(a_t^i|s_t^i)}{\partial\rho}(z_t^i - v(s_t^i))$, using the REINFORCE algorithm[25] with baseline $v(s_t^i)$ for variance reduction. On the first pass through the training pipeline, the baseline was set to zero; on the second pass we used the value network $v_\theta(s)$ as a baseline; this provided a small performance boost. The policy network was trained in this way for 10,000 mini-batches of 128 games, using 50 GPUs, for one day.

Value network: regression. We trained a value network $v_\theta(s) \approx v^{p_\rho}(s)$ to approximate the value function of the RL policy network p_ρ. To avoid overfitting to the strongly correlated positions within games, we constructed a new data set of uncorrelated self-play positions. This data set consisted of over 30 million positions, each drawn from a unique game of self-play. Each game was generated in three phases by randomly sampling a time step $U \sim \text{unif}\{1, 450\}$, and sampling the first $t = 1, \ldots U-1$ moves from the SL policy network, $a_t \sim p_\sigma(\cdot|s_t)$; then sampling one move uniformly at random from available moves, $a_U \sim \text{unif}\{1, 361\}$ (repeatedly until a_U is legal); then sampling the remaining sequence of moves until the game terminates, $t = U+1, \ldots T$, from the RL policy network, $a_t \sim p_\rho(\cdot|s_t)$. Finally, the game is scored to determine the outcome $z_t = \pm r(s_T)$. Only a single training example (s_{U+1}, z_{U+1}) is added to the data set from each game. This data provides unbiased samples of the value function $v^{p_\rho}(s_{U+1}) = \mathbb{E}[z_{U+1}|s_{U+1}, a_{U+1,\ldots T} \sim p_\rho]$. During the first two phases of generation we sample from noisier distributions so as to increase the diversity of the data set. The training method was identical to SL policy network training, except that the parameter update was based on mean squared error between the predicted values and the observed rewards, $\Delta\theta = \frac{\alpha}{m}\sum_{k=1}^{m}(z^k - v_\theta(s^k))\frac{\partial v_\theta(s^k)}{\partial\theta}$. The value network was trained for 50 million mini-batches of 32 positions, using 50 GPUs, for one week.

Features for policy/value network. Each position s was pre-processed into a set of 19×19 feature planes. The features that we use come directly from the raw representation of the game rules, indicating the status of each intersection of the Go board: stone colour, liberties (adjacent empty points of stone's chain), captures, legality, turns since stone was played, and (for the value network only) the current colour to play. In addition, we use one simple tactical feature that computes the outcome of a ladder search[7]. All features were computed relative to the current colour to play; for example, the stone colour at each intersection was represented as either player or opponent rather than black or white. Each integer feature value is split into multiple 19×19 planes of binary values (one-hot encoding). For example, separate binary feature planes are used to represent whether an intersection has 1 liberty, 2 liberties,…, $\geq$8 liberties. The full set of feature planes are listed in Extended Data Table 2.

Neural network architecture. The input to the policy network is a $19 \times 19 \times 48$ image stack consisting of 48 feature planes. The first hidden layer zero pads the input into a 23×23 image, then convolves k filters of kernel size 5×5 with stride 1 with the input image and applies a rectifier nonlinearity. Each of the subsequent hidden layers 2 to 12 zero pads the respective previous hidden layer into a 21×21 image, then convolves k filters of kernel size 3×3 with stride 1, again followed by a rectifier nonlinearity. The final layer convolves 1 filter of kernel size 1×1 with stride 1, with a different bias for each position, and applies a softmax function. The match version of AlphaGo used $k = 192$ filters; Fig. 2b and Extended Data Table 3 additionally show the results of training with $k = 128$, 256 and 384 filters.

The input to the value network is also a $19 \times 19 \times 48$ image stack, with an additional binary feature plane describing the current colour to play. Hidden layers 2 to 11 are identical to the policy network, hidden layer 12 is an additional convolution layer, hidden layer 13 convolves 1 filter of kernel size 1×1 with stride 1, and hidden layer 14 is a fully connected linear layer with 256 rectifier units. The output layer is a fully connected linear layer with a single tanh unit.

Evaluation. We evaluated the relative strength of computer Go programs by running an internal tournament and measuring the Elo rating of each program. We estimate the probability that program a will beat program b by a logistic function $p(a \text{ beats } b) = \frac{1}{1 + \exp(c_{\text{elo}}(e(b) - e(a))}$, and estimate the ratings $e(\cdot)$ by Bayesian logistic regression, computed by the *BayesElo* program[37] using the standard constant $c_{\text{elo}} = 1/400$. The scale was anchored to the *BayesElo* rating of professional

© 2016 Macmillan Publishers Limited. All rights reserved

如果最大化访问次数的行动与最大化行动价值的行动不一致。时间控制旨在将大部分时间用于中局。当 AlphaGo 的整体评估值低于估计的 10%的获胜概率时，即 () < −，AlphaGo 认输。

最大限度 a 0 8.

AlphaGo 并未采用大多数蒙特卡洛围棋程序中使用的“所有走法优先”或“快速行动值估计”启发式方法；当使用策略网络作为先验知识时，这些有偏差的启发式方法似乎并没有带来任何额外好处。此外，AlphaGo 也未使用渐进式扩展、动态协和或开局库。AlphaGo 在樊麾对局中使用的参数列于扩展数据表 5 中。展开策略。展开策略（|）

pas 是一种基于快速增量计算的局部模式特征的线性 softmax 策略。这些特征包括围绕导致当前状态 s 的前一步棋的“响应”模式，以及围绕状态 s 中候选走法 a 的“非响应”模式。每个非响应模式都是一个二元特征，匹配以 a 为中心的特定 3 × 3 模式，该模式由每个相邻交点的颜色（黑色、白色、空白）和自由度（1、2、≥3）定义。每个响应模式都是一个二元特征，匹配以前一步棋为中心的 12 点菱形模式的颜色和自由度。此外，少量手工设计的局部特征编码了常识性的围棋规则（参见扩展数据表 4）。与策略网络类似，展开策略的权重 π 通过随机梯度下降法，利用 Tygem 服务器上 800 万个人类对局的局面进行训练，以最大化对数似然。在空板上，每个 CPU 线程每秒大约执行 1,000 次模拟。

我们的展开策略 p(a|s) 包含的手工知识比最先进的围棋程序要少。相反，我们利用蒙特卡洛树搜索算法 (MCTS) 中更高质量的动作选择，该算法同时参考了搜索树和策略网络。我们引入了一种新技术，该技术缓存搜索树中的所有走法，然后在展开过程中执行类似的走法；这是“最后好回复”启发式算法的推广。在遍历树的每一步，最有可能的走法以及围绕前一步和当前走法的 3×3 模式上下文（颜色、自由度和石子数）被插入到哈希表中。在展开的每一步，模式上下文与哈希表进行匹配；如果找到匹配项，则以高概率执行存储的走法。

对称性。在之前的研究中，人们利用围棋的对称性，在卷积层中使用旋转不变和反射不变的滤波器。虽然这种方法在小型神经网络中可能有效，但实际上会降低大型网络的性能，因为它会阻止中间滤波器识别特定的非对称模式。相反，我们通过在运行时动态变换每个位置 s 来利用对称性，变换过程使用由八个反射和旋转构成的二面体群 d(s), ⋯, d(s)。在一个显式对称性集成中，所有 8 个位置的小批量数据被传递到策略网络或值网络，并并行计算。对于值网络，输出值只需取平均值即可。

对于策略网络，输出概率平面被旋转/反射回原始方向，并取平均值以提供集成预测，(·|) = ∑ ((·| ( )))

这种方法用于我们的原始网络评估 p（参见扩展数据表 3）p 而 APV-MCTS 则使用隐式对称集成，每次评估随机选择一个旋转/反射 j ∈ [1, 8]。我们仅针对该方向计算一次评估；在每次模拟中，我们通过 v(d(s)) 计算叶节点 s 的值，并允许搜索过程对这些评估结果取平均值。类似地，我们针对单个随机选择的旋转/反射 ((·| ( ))) 计算策略网络。

d_j^{-1} p d s .

策略网络：分类。我们训练了策略网络 P，使其能够根据 KGS 数据集中的专家走法对棋局进行分类。该数据集包含 2940 万个棋局，这些棋局来自 16 万盘由 KGS 6 段至 9 段棋手对弈的对局；其中 35.4% 为让子局。数据集被分为测试集（前 100 万个棋局）和训练集（剩余的 2840 万个棋局）。数据集中排除了平局走法。每个棋局包含原始棋盘描述 s 和棋手选择的走法 a。我们对数据集进行了扩充，使其包含每个棋局的所有八个反射和旋转。每个棋局的对称性增强和输入特征均已预先计算。在每个训练步骤中，我们从扩充后的 KGS 数据集中随机抽取 m 个样本的小批量。

1 并应用异步随机梯度下降更新来最大化动作的对数似然，Δσ =

步长 α 初始值为 0.003，每 8000 万个训练步减半，不包含动量项，小批量大小为 m = 16。使用 DistBelief 在 50 个 GPU 上异步更新；丢弃超过 100 步的梯度。训练耗时约 3 周，共 3.4 亿个训练步。

策略网络：强化学习。我们进一步采用策略梯度强化学习训练策略网络。每次迭代包含 n 个并行进行的博弈小批量，博弈在当前正在训练的策略网络 p 和使用前一次迭代参数 ρ 的对手 p 之间进行。对手 p 是从对手池中随机抽取的，以提高训练的稳定性。权重初始化为 ρ = ρ = σ。每 500 次迭代，我们将当前参数 ρ 添加到对手池中。小批量中的每个博弈 i 都进行到步骤 T 终止，然后进行评分以确定结果。

从每个玩家的角度出发，计算 T。然后重玩这些游戏以确定策略梯度更新，Δρ = ∑ ∑ (− ( ))

使用 REINFORCE 算法，并以基线 ( )vs 进行方差缩减。在训练流程的第一次迭代中，基线设置为零；在第二次迭代中，我们使用价值网络 v(s) 作为基线；这带来了一定的性能提升。策略网络以这种方式训练了 10,000 个包含 128 个游戏的小批量，使用 50 个 GPU，训练时间为一天。价值网络：回归。我们训练了一个价值网络 ( ) ≈ ( )。

为了近似强化学习策略网络 p 的价值函数，我们构建了一个新的不相关的自博弈位置数据集。该数据集包含超过 3000 万个位置，每个位置均来自一场独特的自博弈。每场博弈的生成分为三个阶段：首先，随机抽取一个时间步长 U ~ unif{1, 450}，并从自博弈策略网络 a~ p(·|s) 中抽取前 t = 1,⋯ U − 1 步；然后，从可用步数 a ~ unif{1, 361} 中均匀随机抽取一步（重复此过程直至 a 合法）；最后，从强化学习策略网络 a~ p(·|s) 中抽取剩余的步数序列直至博弈结束，t = U + 1,⋯ T。最后，对博弈进行评分以确定结果 z= ±r(s)。每场博弈仅向数据集中添加一个训练样本 (s, z)。这些数据提供了值函数 E 的无偏样本。

在生成的前两个阶段，我们从噪声较大的分布中采样，以增加数据集的多样性。训练方法与 SL 策略网络训练相同，只是参数更新基于预测值和观测奖励之间的均方误差。

这个值网络使用 50 个 GPU，经过一周时间，训练了 5000 万个 32 个位置的小批量数据。

策略/价值网络的特征。每个位置 s 都被预处理成一组 19 × 19 的特征平面。我们使用的特征直接来源于游戏规则的原始表示，指示围棋棋盘上每个交叉点的状态：棋子颜色、自由（棋子链上相邻的空位）、吃子、合法性、自该棋子落子以来的回合数，以及（仅针对价值网络）当前轮次。此外，我们还使用一个简单的战术特征来计算梯子搜索的结果。所有特征的计算都与当前轮次相关；例如，每个交叉点的棋子颜色表示为玩家或对手，而不是黑色或白色。每个整数特征值被分割成多个 19 × 19 的二进制值平面（独热编码）。例如，使用单独的二进制特征平面来表示交叉点是否具有 1 个自由、2 个自由、⋯⋯、≥8 个自由。完整的特征平面集列于扩展数据表 2 中。

神经网络架构。策略网络的输入是一个 19 × 19 × 48 的图像堆栈，包含 48 个特征平面。第一个隐藏层将输入图像零填充为 23 × 23 的图像，然后使用 k 个步长为 1、核大小为 5 × 5 的滤波器与输入图像进行卷积，并应用整流器非线性函数。接下来的第 2 至 12 个隐藏层分别将前一个隐藏层的图像零填充为 21 × 21 的图像，然后使用步长为 1、核大小为 3 × 3 的滤波器进行卷积，同样应用整流器非线性函数。最后一层使用步长为 1、核大小为 1 × 1 的滤波器进行卷积，每个位置的偏置不同，并应用 softmax 函数。AlphaGo 的比赛版本使用了 k = 192 个滤波器；图 2b 和扩展数据表 3 还展示了使用 k = 128、256 和 384 个滤波器进行训练的结果。

值网络的输入也是一个 19 × 19 × 48 的图像堆栈，并附加一个二值特征平面来描述当前要播放的颜色。隐藏层 2 到 11 与策略网络相同，隐藏层 12 是一个额外的卷积层，隐藏层 13 使用步长为 1 的 1 × 1 卷积核进行卷积，隐藏层 14 是一个包含 256 个整流单元的全连接线性层。输出层是一个包含单个 tanh 单元的全连接线性层。

评估。我们通过举办内部比赛并测量每个程序的 Elo 等级分来评估计算机围棋程序的相对实力。我们使用逻辑函数估计程序 a 战胜程序 b 的概率。

帕(节拍 b) = 并使用贝叶斯方法估计评分 e(·)

逻辑回归，由 BayesElo 程序计算，使用标准常数 c= 1/400。该量表锚定于 BayesElo 专业评级。

© 2016 麦克米伦出版有限公司。版权所有。

Go player Fan Hui (2,908 at date of submission)[62]. All programs received a maximum of 5 s computation time per move; games were scored using Chinese rules with a *komi* of 7.5 points (extra points to compensate white for playing second). We also played handicap games where AlphaGo played white against existing Go programs; for these games we used a non-standard handicap system in which *komi* was retained but black was given additional stones on the usual handicap points. Using these rules, a handicap of K stones is equivalent to giving $K-1$ free moves to black, rather than $K-1/2$ free moves using standard no-*komi* handicap rules. We used these handicap rules because AlphaGo's value network was trained specifically to use a *komi* of 7.5.

With the exception of distributed AlphaGo, each computer Go program was executed on its own single machine, with identical specifications, using the latest available version and the best hardware configuration supported by that program (see Extended Data Table 6). In Fig. 4, approximate ranks of computer programs are based on the highest KGS rank achieved by that program; however, the KGS version may differ from the publicly available version.

The match against Fan Hui was arbitrated by an impartial referee. Five formal games and five informal games were played with 7.5 *komi*, no handicap, and Chinese rules. AlphaGo won these games 5–0 and 3–2 respectively (Fig. 6 and Extended Data Table 1). Time controls for formal games were 1 h main time plus three periods of 30 s *byoyomi*. Time controls for informal games were three periods of 30 s *byoyomi*. Time controls and playing conditions were chosen by Fan Hui in advance of the match; it was also agreed that the overall match outcome would be determined solely by the formal games. To approximately assess the relative rating of Fan Hui to computer Go programs, we appended the results of all ten games to our internal tournament results, ignoring differences in time controls.

39. Littman, M. L. Markov games as a framework for multi-agent reinforcement learning. In *11th International Conference on Machine Learning*, 157–163 (1994).
40. Knuth, D. E. & Moore, R. W. An analysis of alpha-beta pruning. *Artif. Intell.* **6,** 293–326 (1975).
41. Sutton, R. Learning to predict by the method of temporal differences. *Mach. Learn.* **3,** 9–44 (1988).
42. Baxter, J., Tridgell, A. & Weaver, L. Learning to play chess using temporal differences. *Mach. Learn.* **40,** 243–263 (2000).
43. Veness, J., Silver, D., Blair, A. & Uther, W. Bootstrapping from game tree search. In *Advances in Neural Information Processing Systems* (2009).
44. Samuel, A. L. Some studies in machine learning using the game of checkers II - recent progress. *IBM J. Res. Develop.* **11,** 601–617 (1967).
45. Schaeffer, J., Hlynka, M. & Jussila, V. Temporal difference learning applied to a high-performance game-playing program. In *17th International Joint Conference on Artificial Intelligence*, 529–534 (2001).
46. Tesauro, G. TD-gammon, a self-teaching backgammon program, achieves master-level play. *Neural Comput.* **6,** 215–219 (1994).
47. Dahl, F. Honte, a Go-playing program using neural nets. In *Machines that learn to play games*, 205–223 (Nova Science, 1999).
48. Rosin, C. D. Multi-armed bandits with episode context. *Ann. Math. Artif. Intell.* **61,** 203–230 (2011).
49. Lanctot, M., Winands, M. H. M., Pepels, T. & Sturtevant, N. R. Monte Carlo tree search with heuristic evaluations using implicit minimax backups. In *IEEE Conference on Computational Intelligence and Games*, 1–8 (2014).
50. Gelly, S., Wang, Y., Munos, R. & Teytaud, O. Modification of UCT with patterns in Monte-Carlo Go. Tech. Rep. 6062, INRIA (2006).
51. Silver, D. & Tesauro, G. Monte-Carlo simulation balancing. In *26th International Conference on Machine Learning*, 119 (2009).
52. Huang, S.-C., Coulom, R. & Lin, S.-S. Monte-Carlo simulation balancing in practice. In *7th International Conference on Computers and Games*, 81–92 (Springer-Verlag, 2011).
53. Baier, H. & Drake, P. D. The power of forgetting: improving the last-good-reply policy in Monte Carlo Go. *IEEE Trans. Comput. Intell. AI in Games* **2,** 303–309 (2010).
54. Huang, S. & Müller, M. Investigating the limits of Monte-Carlo tree search methods in computer Go. In *8th International Conference on Computers and Games*, 39–48 (2013).
55. Segal, R. B. On the scalability of parallel UCT. *Computers and Games* **6515,** 36–47 (2011).
56. Enzenberger, M. & Müller, M. A lock-free multithreaded Monte-Carlo tree search algorithm. In *12th Advances in Computer Games Conference*, 14–20 (2009).
57. Huang, S.-C., Coulom, R. & Lin, S.-S. Time management for Monte-Carlo tree search applied to the game of Go. In *International Conference on Technologies and Applications of Artificial Intelligence*, 462–466 (2010).
58. Gelly, S. & Silver, D. Monte-Carlo tree search and rapid action value estimation in computer Go. *Artif. Intell.* **175,** 1856–1875 (2011).
59. Baudiš, P. Balancing MCTS by dynamically adjusting the komi value. *ICGA J.* **34,** 131 (2011).
60. Baier, H. & Winands, M. H. Active opening book application for Monte-Carlo tree search in 19×19 Go. In *Benelux Conference on Artificial Intelligence*, 3–10 (2011).
61. Dean, J. *et al.* Large scale distributed deep networks. In *Advances in Neural Information Processing Systems*, 1223–1231 (2012).
62. Go ratings. http://www.goratings.org.

© 2016 Macmillan Publishers Limited. All rights reserved

围棋选手樊麾（提交时等级分 2908）。所有程序每步棋的计算时间上限为 5 秒；比赛采用中国式规则，贴目 7.5 分（额外加分以补偿白方后手）。我们也进行了让子对局，AlphaGo 执白对阵现有的围棋程序；这些对局采用了一种非标准的让子规则，保留了贴目，但黑方在通常的让子点数上额外获得子。根据这种规则，K 子让子相当于给黑方 K-1 步，而不是像标准无贴目让子规则那样给黑方 K-1/2 步。我们使用这种让子规则是因为 AlphaGo 的价值网络经过专门训练，能够适应 7.5 贴目。

除了分布式 AlphaGo 之外，每个计算机围棋程序都在各自独立的机器上运行，配置完全相同，并使用了该程序支持的最新版本和最佳硬件配置（参见扩展数据表 6）。图 4 中，计算机程序的近似排名基于该程序达到的最高 KGS 排名；但是，KGS 版本可能与公开版本有所不同。

与樊麾的比赛由一位公正的裁判进行裁决。比赛共进行了五盘正式对局和五盘非正式对局，采用 7.5 贴目、无让子、中国式规则。AlphaGo 分别以 5-0 和 3-2 的比分赢得了这些比赛（图 6 和扩展数据表 1）。正式对局的时限为 1 小时主时，外加三个 30 秒读秒的读时。非正式对局的时限为三个 30 秒读时。时限和比赛条件均由樊麾在赛前选定；双方还约定，比赛的最终结果将完全由正式对局的成绩决定。为了大致评估樊麾相对于计算机围棋程序的水平，我们将所有十盘对局的结果添加到我们的内部比赛结果中，忽略了时限的差异。

39. Littman，《ML 马尔可夫博弈作为多智能体强化学习的框架》学习。第 11 届国际机器学习会议，157–163 (1994)。

40. Knuth, DE & Moore, RW 对 alpha-beta 剪枝的分析。人工智能 6, 293–326 (1975)。

41. Sutton, R. 通过时间差分法学习预测。机器学习 3, 9–44 (1988)。
42. Baxter, J., Tridgell, A. & Weaver, L. 使用时间差分学习下国际象棋。机器学习 40, 243–263 (2000)。
43. Veness, J.、Silver, D.、Blair, A. 和 Uther, W. 从博弈树搜索中引导。在神经信息处理系统进展（2009）中。

44. Samuel, AL. 使用跳棋游戏进行机器学习的一些研究 II - 最新进展。IBM 研究与开发杂志 11, 601–617 (1967)。
45. Schaeffer, J.、Hlynka, M. 和 Jussila, V. 时间差分学习应用于高性能游戏程序。第 17 届国际人工智能联合会议，529–534 (2001)。

46. Tesauro, G. TD-gammon，一个自学双陆棋程序，达到了大师级水平。神经计算 6, 215–219 (1994)。
47. Dahl, F. Honte，使用神经网络的围棋程序。载于《学习玩游戏的机器》，205–223（Nova Science，1999 年）。
48. Rosin, CD 多臂老虎机与事件上下文。数学与人工智能年鉴。61, 203–230 (2011)。
49. Lanctot, M.、Winands, MHM、Pepels, T. 和 Sturtevant, NR，使用隐式极小极大备份的启发式评估的蒙特卡洛树搜索。在 IEEE 计算智能与游戏会议论文集，1–8 (2014)。

50. Gelly, S.、Wang, Y.、Munos, R. 和 Teytaud, O.，《蒙特卡罗围棋中模式的 UCT 修改》。技术报告 6062，INRIA（2006 年）。
51. Silver, D. 和 Tesauro, G. Monte-Carlo 模拟平衡。在第 26 届国际机器学习会议上，119 (2009)。
52. Huang, S.-C., Coulom, R. & Lin, S.-S. Monte-Carlo simulation balance in practice. In 7th International Conference on Computers and Games, 81–92 (Springer-Verlag, 2011).
53. Baier, H. & Drake, PD 遗忘的力量：改进最后一次良好回复蒙特卡洛围棋中的策略。IEEE Trans. Comput. Intell. AI in Games 2, 303–309 (2010)。

54. Huang, S. 和 Müller, M. 探讨蒙特卡洛树搜索的局限性计算机围棋方法。第 8 届国际计算机与游戏会议，39–48 (2013)。

55. Segal, RB 关于并行 UCT 的可扩展性。计算机与游戏 6515, 36–47 (2011)。

56. Enzenberger, M. 和 Müller, M. 提出了一种无锁多线程蒙特卡洛树。搜索算法。第 12 届计算机游戏进展会议，14–20 (2009)。

57. 黄世昌、库洛姆、林世森，《蒙特卡洛树的时间管理》搜索应用于围棋游戏。在人工智能技术与应用国际会议上，462–466 (2010)。

58. Gelly, S. & Silver, D. Monte-Carlo 树搜索和计算机围棋中的快速动作值估计。人工智能 175, 1856–1875 (2011)。
59. Baudiš, P. 通过动态调整贴目值来平衡 MCTS。ICGA J. 34, 131 (2011).
60. Baier, H. & Winands, MH 蒙特卡洛主动开账簿应用程序 19×19 Go 中的树搜索。在 Benelux 人工智能会议上，3–10 (2011)。

61. Dean, J. 等人。大规模分布式深度网络。神经信息处理系统进展，1223–1231 (2012)。

62. 前往评分。http://www.goratings.org.

© 2016 麦克米伦出版有限公司。版权所有。

Extended Data Table 1 | Details of match between AlphaGo and Fan Hui

Date	Black	White	Category	Result
5/10/15	Fan Hui	*AlphaGo*	Formal	*AlphaGo* wins by 2.5 points
5/10/15	Fan Hui	*AlphaGo*	Informal	Fan Hui wins by resignation
6/10/15	*AlphaGo*	Fan Hui	Formal	*AlphaGo* wins by resignation
6/10/15	*AlphaGo*	Fan Hui	Informal	*AlphaGo* wins by resignation
7/10/15	Fan Hui	*AlphaGo*	Formal	*AlphaGo* wins by resignation
7/10/15	Fan Hui	*AlphaGo*	Informal	*AlphaGo* wins by resignation
8/10/15	*AlphaGo*	Fan Hui	Formal	*AlphaGo* wins by resignation
8/10/15	*AlphaGo*	Fan Hui	Informal	*AlphaGo* wins by resignation
9/10/15	Fan Hui	*AlphaGo*	Formal	*AlphaGo* wins by resignation
9/10/15	*AlphaGo*	Fan Hui	Informal	Fan Hui wins by resignation

The match consisted of five formal games with longer time controls, and five informal games with shorter time controls. Time controls and playing conditions were chosen by Fan Hui in advance of the match.

© 2016 Macmillan Publishers Limited. All rights reserved

扩展数据表 1 | AlphaGo 与樊麾比赛详情

日期	黑白组	类别	结果
2015 年 5 月 10 日	樊麾	AlphaGo 正式比赛	AlphaGo 以 2.5 分获胜
2015 年 5 月 10 日	樊麾	AlphaGo 非正式比赛	樊麾因对手弃权获胜
2015 年 6 月 10 日	AlphaGo	樊麾 正式比赛	AlphaGo 因对手弃权获胜
2015 年 6 月 10 日	AlphaGo	樊麾 非正式比赛	AlphaGo 因对手弃权获胜
2015 年 7 月 10 日	樊麾	AlphaGo 正式比赛	AlphaGo 因对手弃权获胜
2015 年 7 月 10 日	樊麾	AlphaGo 非正式比赛	AlphaGo 因对手弃权获胜
2015 年 8 月 10 日	AlphaGo	樊麾 正式比赛	AlphaGo 因对手弃权获胜
2015 年 8 月 10 日	AlphaGo	樊麾 非正式比赛	AlphaGo 因对手弃权获胜
2015 年 9 月 10 日	樊麾	AlphaGo 正式比赛	AlphaGo 因对手弃权获胜
2015 年 9 月 10 日	AlphaGo	樊麾 非正式比赛	樊麾因辞职而获胜

比赛包括五场正式比赛（时限较长）和五场非正式比赛（时限较短）。

比赛的时间限制和比赛条件都是樊麾在赛前选定的。

© 2016 麦克米伦出版有限公司。版权所有。

Extended Data Table 2 | Input features for neural networks

Feature	# of planes	Description
Stone colour	3	Player stone / opponent stone / empty
Ones	1	A constant plane filled with 1
Turns since	8	How many turns since a move was played
Liberties	8	Number of liberties (empty adjacent points)
Capture size	8	How many opponent stones would be captured
Self-atari size	8	How many of own stones would be captured
Liberties after move	8	Number of liberties after this move is played
Ladder capture	1	Whether a move at this point is a successful ladder capture
Ladder escape	1	Whether a move at this point is a successful ladder escape
Sensibleness	1	Whether a move is legal and does not fill its own eyes
Zeros	1	A constant plane filled with 0
Player color	1	Whether current player is black

Feature planes used by the policy network (all but last feature) and value network (all features).

© 2016 Macmillan Publishers Limited. All rights reserved

扩展数据表 2 | 神经网络的输入特征

特征	飞机数量	描述
石头颜色	3	玩家棋子/对手棋子/空位
一	1	一个充满 1 的恒定平面
自	8	自上次走棋以来，已经进行了多少回合?
自由	8	自由点数（相邻空点数）
捕获大小	8	会俘获多少对方的棋子?
自 Atari 尺寸	8	会有多少自己的石头被俘获
走棋后的自由度 8 此步棋之后的自由度数 梯子吃子 1 此时的走棋是否成功吃子 梯子逃脱 1 此时的走棋是否成功逃脱 合理性 1 走棋是否合法且不自满 零 1 一个充满 0 的恒定平面		
球员颜色	1	当前球员是否为黑人

策略网络（除最后一个特征外的所有特征）和价值网络（所有特征）使用的特征平面。

© 2016 麦克米伦出版有限公司。版权所有。

Extended Data Table 3 | Supervised learning results for the policy network

Architecture			Evaluation				
Filters	Symmetries	Features	Test accuracy %	Train accuracy %	Raw net wins %	*AlphaGo* wins %	Forward time (ms)
128	1	48	54.6	57.0	36	53	2.8
192	1	48	55.4	58.0	50	50	4.8
256	1	48	55.9	59.1	67	55	7.1
256	2	48	56.5	59.8	67	38	13.9
256	4	48	56.9	60.2	69	14	27.6
256	8	48	57.0	60.4	69	5	55.3
192	1	4	47.6	51.4	25	15	4.8
192	1	12	54.7	57.1	30	34	4.8
192	1	20	54.7	57.2	38	40	4.8
192	8	4	49.2	53.2	24	2	36.8
192	8	12	55.7	58.3	32	3	36.8
192	8	20	55.8	58.4	42	3	36.8

The policy network architecture consists of 128, 192 or 256 filters in convolutional layers; an explicit symmetry ensemble over 2, 4 or 8 symmetries; using only the first 4, 12 or 20 input feature planes listed in Extended Data Table 1. The results consist of the test and train accuracy on the KGS data set; and the percentage of games won by given policy network against AlphaGo's policy network (highlighted row 2): using the policy networks to select moves directly (raw wins); or using AlphaGo's search to select moves (AlphaGo wins); and finally the computation time for a single evaluation of the policy network.

© 2016 Macmillan Publishers Limited. All rights reserved

扩展数据表 3 | 策略网络的监督学习结果

建筑学			评估				
过滤器	对称性	特征	测试 准确度	性感 %	火车准确-性感 %	原始网络胜率	AlphaGo 胜率	向前时间（毫秒）
128	1	48	54.6	57.0	36	53	2.8
192	1	48	55.4	58.0	50	50	4.8
256	1	48	55.9	59.1	67	55	7.1
256	2	48	56.5	59.8	67	38	13.9
256	4	48	56.9	60.2	69	14	27.6
256	8	48	57.0	60.4	69	5	55.3
192	1	4	47.6	51.4	25	15	4.8
192	1	12	54.7	57.1	30	34	4.8
192	1	20	54.7	57.2	38	40	4.8
192	8	4	49.2	53.2	24	2	36.8
192	8	12	55.7	58.3	32	3	36.8
192	8	20	55.8	58.4	42	3	36.8

策略网络架构由 128、192 或 256 个卷积层滤波器组成；显式对称集成，涵盖 2、4 或 8 个对称性；仅使用扩展数据表 1 中列出的前 4、12 或 20 个输入特征平面。结果包括 KGS 数据集上的测试和训练准确率；以及给定策略网络相对于 AlphaGo 策略网络的胜率（第 2 行高亮显示）：直接使用策略网络选择走法（原始胜率）；或使用 AlphaGo 的搜索算法选择走法（AlphaGo 胜率）；最后是策略网络单次评估的计算时间。

© 2016 麦克米伦出版有限公司。版权所有。

Extended Data Table 4 | Input features for rollout and tree policy

Feature	# of patterns	Description
Response	1	Whether move matches one or more response pattern features
Save atari	1	Move saves stone(s) from capture
Neighbour	8	Move is 8-connected to previous move
Nakade	8192	Move matches a *nakade* pattern at captured stone
Response pattern	32207	Move matches 12-point diamond pattern near previous move
Non-response pattern	69338	Move matches 3×3 pattern around move
Self-atari	1	Move allows stones to be captured
Last move distance	34	Manhattan distance to previous two moves
Non-response pattern	32207	Move matches 12-point diamond pattern centred around move

Features used by the rollout policy (first set) and tree policy (first and second set). Patterns are based on stone colour (black/white/empty) and liberties (1, 2, $\geq$3) at each intersection of the pattern.

© 2016 Macmillan Publishers Limited. All rights reserved

扩展数据表 4 | 推广和树状策略的输入特征

Feature	# of patterns	Description
Response	1	Whether move matches one or more response pattern features
Save atari	1	Move saves stone(s) from capture
Neighbour	8	Move is 8-connected to previous move
Nakade	8192	Move matches a *nakade* pattern at captured stone
Response pattern	32207	Move matches 12-point diamond pattern near previous move
Non-response pattern	69338	Move matches 3×3 pattern around move
Self-atari	1	Move allows stones to be captured
Last move distance	34	Manhattan distance to previous two moves
Non-response pattern	32207	Move matches 12-point diamond pattern centred around move

展开策略（第一组）和树状策略（第一组和第二组）所使用的特征。模式基于石头颜色（黑色/白色/空白）以及模式每个交叉点的自由度（1、2、≥3）。

© 2016 麦克米伦出版有限公司。版权所有。

Extended Data Table 5 | Parameters used by AlphaGo

Symbol	Parameter	Value
β	Softmax temperature	0.67
λ	Mixing parameter	0.5
n_{vl}	Virtual loss	3
n_{thr}	Expansion threshold	40
c_{puct}	Exploration constant	5

© 2016 Macmillan Publishers Limited. All rights reserved

扩展数据表 5 | AlphaGo 使用的参数

象征	范围	价值
β	Softmax 温度	0.67
λ 混合参数		0.5
虚拟损失		3
n 扩展阈值		40
c	探索常数	5

© 2016 麦克米伦出版有限公司。版权所有。

Extended Data Table 6 | Results of a tournament between different Go programs

Short name	Computer Player	Version	Time settings	CPUs	GPUs	KGS Rank	Elo
α^d_{rvp}	Distributed AlphaGo	See Methods	5 seconds	1202	176	–	3140
α_{rvp}	AlphaGo	See Methods	5 seconds	48	8	–	2890
CS	CrazyStone	2015	5 seconds	32	–	6d	1929
ZN	Zen	5	5 seconds	8	–	6d	1888
PC	Pachi	10.99	400,000 sims	16	–	2d	1298
FG	Fuego	svn1989	100,000 sims	16	–	–	1148
GG	GnuGo	3.8	level 10	1	–	5k	431
CS_4	CrazyStone	4 handicap stones	5 seconds	32	–	–	2526
ZN_4	Zen	4 handicap stones	5 seconds	8	–	–	2413
PC_4	Pachi	4 handicap stones	400,000 sims	16	–	–	1756

Each program played with a maximum of 5 s thinking time per move; the games against Fan Hui were conducted using longer time controls, as described in Methods. CN_4, ZN_4 and PC_4 were given 4 handicap stones; *komi* was 7.5 in all games. Elo ratings were computed by BayesElo.

© 2016 Macmillan Publishers Limited. All rights reserved

扩展数据表 6 | 不同围棋程序之间比赛的结果

简称	电脑玩家	版本	时间设置	CPU	GPU	KGS 排名	埃洛
α	分布式 AlphaGo	参见“方法”部分	5 秒	1202	176	–	3140
α	AlphaGo	参见“方法”部分	5 秒	48	8	–	2890
CS	疯狂石头	2015	5 秒	32	–	6 天	1929
ZN	禅	5	5 秒	8	–	6 天	1888
个人电脑	帕奇	10.99	40 万模拟市民	16	–	2d	1298
前锋	火	svn1989	10 万模拟市民	16	–	–	1148
GG	GnuGo	3.8	10 级	1	–	5 公里	431
CS	疯狂石头	4 块障碍石	5 秒	32	–	–	2526
ZN	禅	4 块障碍石	5 秒	8	–	–	2413
个人电脑	帕奇	4 块障碍石	40 万模拟市民	16	–	–	1756

每个程序每步棋的思考时间最多为 5 秒；与樊麾的对局则采用更长的思考时间，具体方法见“方法”部分。CN、ZN 和 PC 均让 4 子；所有对局的贴目数均为 7.5。等级分由 BayesElo 计算。

© 2016 麦克米伦出版有限公司。版权所有。

Extended Data Table 7 | Results of a tournament between different variants of AlphaGo

Short name	Policy network	Value network	Rollouts	Mixing constant	Policy GPUs	Value GPUs	Elo rating
α_{rvp}	p_σ	v_θ	p_π	$\lambda = 0.5$	2	6	2890
α_{vp}	p_σ	v_θ	–	$\lambda = 0$	2	6	2177
α_{rp}	p_σ	–	p_π	$\lambda = 1$	8	0	2416
α_{rv}	$[p_\tau]$	v_θ	p_π	$\lambda = 0.5$	0	8	2077
α_v	$[p_\tau]$	v_θ	–	$\lambda = 0$	0	8	1655
α_r	$[p_\tau]$	–	p_π	$\lambda = 1$	0	0	1457
α_p	p_σ	–	–	–	0	0	1517

Evaluating positions using rollouts only (α_{rp}, α_r), value nets only (α_{vp}, α_v), or mixing both (α_{rvp}, α_{rv}); either using the policy network $p_\sigma(\alpha_{rvp}, \alpha_{vp}, \alpha_{rp})$, or no policy network ($\alpha_{rvp}$, α_{vp}, α_{rp}), that is, instead using the placeholder probabilities from the tree policy p_τ throughout. Each program used 5 s per move on a single machine with 48 CPUs and 8 GPUs. Elo ratings were computed by BayesElo.

© 2016 Macmillan Publishers Limited. All rights reserved

扩展数据表 7 | 不同 AlphaGo 变体之间的比赛结果

短期策略值发布混合策略值 Elo 名称网络网络常量 GPUs GPUs							
							等级
α	p	v	p	λ = 0.5	2	6	2890
α	p	v	–	λ = 0	2	6	2177
α	p	–	p	λ = 1	8	0	2416
α	[p]	v	p	λ = 0.5	0	8	2077
α	[p]	v	–	λ = 0	0	8	1655
α	[p]	–	p	λ = 1	0	0	1457
α	p	–	–	–	0	0	1517

仅使用展开式（α, α）、仅使用价值网络（α, α）或两者混合（α, α）来评估局面；使用策略网络 p(α, α, α) 或不使用策略网络（α, α, α），即使用树策略 pthroughout 中的占位符概率。每个程序在一台配备 48 个 CPU 和 8 个 GPU 的机器上，每步棋耗时 5 秒。Elo 等级分由 BayesElo 计算。

© 2016 麦克米伦出版有限公司。版权所有。

Extended Data Table 8 | Results of a tournament between AlphaGo and distributed AlphaGo, testing scalability with hardware

AlphaGo	Search threads	CPUs	GPUs	Elo
Asynchronous	1	48	8	2203
Asynchronous	2	48	8	2393
Asynchronous	4	48	8	2564
Asynchronous	8	48	8	2665
Asynchronous	16	48	8	2778
Asynchronous	32	48	8	2867
Asynchronous	40	48	8	2890
Asynchronous	40	48	1	2181
Asynchronous	40	48	2	2738
Asynchronous	40	48	4	2850
Distributed	12	428	64	2937
Distributed	24	764	112	3079
Distributed	40	1202	176	3140
Distributed	64	1920	280	3168

Each program played with a maximum of 2 s thinking time per move. Elo ratings were computed by BayesElo.

© 2016 Macmillan Publishers Limited. All rights reserved

扩展数据表 8 | AlphaGo 与分布式 AlphaGo 之间的比赛结果，测试硬件可扩展性

AlphaGo	搜索主题	CPU	GPU	埃洛
异步	1	48	8	2203
异步	2	48	8	2393
异步	4	48	8	2564
异步	8	48	8	2665
异步	16	48	8	2778
异步	32	48	8	2867
异步	40	48	8	2890
异步	40	48	1	2181
异步	40	48	2	2738
异步	40	48	4	2850
分布式	12	428	64	2937
分布式	24	764	112	3079
分布式	40	1202	176	3140
分布式	64	1920	280	3168

每个程序每步棋的思考时间最多为 2 秒。Elo 等级分由 BayesElo 计算得出。

© 2016 麦克米伦出版有限公司。版权所有。

Extended Data Table 9 | Cross-table of win rates in per cent between programs

	α_{rvp}	α_{vp}	α_{rp}	α_{rv}	α_r	α_v	α_p
α_{rvp}	-	1 [0; 5]	5 [4; 7]	0 [0; 4]	0 [0; 8]	0 [0; 19]	0 [0; 19]
α_{vp}	99 [95; 100]	-	61 [52; 69]	35 [25; 48]	6 [1; 27]	0 [0; 22]	1 [0; 6]
α_{rp}	95 [93; 96]	39 [31; 48]	-	13 [7; 23]	0 [0; 9]	0 [0; 22]	4 [1; 21]
α_{rv}	100 [96; 100]	65 [52; 75]	87 [77; 93]	-	0 [0; 18]	29 [8; 64]	48 [33; 65]
α_r	100 [92; 100]	94 [73; 99]	100 [91; 100]	100 [82; 100]	-	78 [45; 94]	78 [71; 84]
α_v	100 [81; 100]	100 [78; 100]	100 [78; 100]	71 [36; 92]	22 [6; 55]	-	30 [16; 48]
α_p	100 [81; 100]	99 [94; 100]	96 [79; 99]	52 [35; 67]	22 [16; 29]	70 [52; 84]	-
CS	100 [97; 100]	74 [66; 81]	98 [94; 99]	80 [70; 87]	5 [3; 7]	36 [16; 61]	8 [5; 14]
ZN	99 [93; 100]	84 [67; 93]	98 [93; 99]	92 [67; 99]	6 [2; 19]	40 [12; 77]	100 [65; 100]
PC	100 [98; 100]	99 [95; 100]	100 [98; 100]	98 [89; 100]	78 [73; 81]	87 [68; 95]	55 [47; 62]
FG	100 [97; 100]	99 [93; 100]	100 [96; 100]	100 [91; 100]	78 [73; 83]	100 [65; 100]	65 [55; 73]
GG	100 [44; 100]	100 [34; 100]	100 [68; 100]	100 [57; 100]	99 [97; 100]	67 [21; 94]	99 [95; 100]
CS_4	77 [69; 84]	12 [8; 18]	53 [44; 61]	15 [8; 24]	0 [0; 3]	0 [0; 30]	0 [0; 8]
ZN_4	86 [77; 92]	25 [16; 38]	67 [56; 76]	14 [7; 27]	0 [0; 12]	0 [0; 43]	-
PC_4	99 [97; 100]	82 [75; 88]	98 [95; 99]	89 [79; 95]	32 [26; 39]	13 [3; 36]	35 [25; 46]

95% Agresti–Coull confidence intervals in grey. Each program played with a maximum of 5 s thinking time per move. CN_4, ZN_4 and PC_4 were given 4 handicap stones; *komi* was 7.5 in all games. Distributed AlphaGo scored 77% [70; 82] against α_{rvp} and 100% against all other programs (no handicap games were played).

© 2016 Macmillan Publishers Limited. All rights reserved

扩展数据表 9 | 各项目胜率交叉表（百分比）

	α	α	α	α	α	α	α
α	-	1	5	0	0	0	0
α	99	-	61	35	6	0	1
α	95	39	-	13	0	0	4
α	100	65	87	-	0	29	48
α	100	94	100	100	-	78	78
α	100	100	100	71	22	-	30
α	100	99	96	52	22	70	-
CS	100	74	98	80	5	36	8
ZN	99	84	98	92	6	40	100
个人电脑	100	99	100	98	78	87	55
前锋	100	99	100	100	78	100	65
GG	100	100	100	100	99	67	99
CS	77	12	53	15	0	0	0
ZN	86	25	67	14	0	0	-
个人电脑	99	82	98	89	32	13	35

图中灰色部分为 95% Agresti-Coull 置信区间。每个程序每步棋的思考时间最多为 5 秒。CN、ZN 和 PC 程序均让 4 子；所有对局的贴目数均为 7.5。分布式 AlphaGo 对α程序的胜率为 77% [70; 82]，对所有其他程序的胜率为 100%（未进行让子对局）。

© 2016 麦克米伦出版有限公司。版权所有。

Extended Data Table 10 | Cross-table of win rates in per cent between programs in the single-machine scalability study

Threads		1	2	4	8	16	32	40	40	40	40
	GPU	8	8	8	8	8	8	8	4	2	1
1	8	-	70 [61;78]	90 [84;94]	94 [83;98]	86 [72;94]	98 [91;100]	98 [92;99]	100 [76;100]	96 [91;98]	38 [25;52]
2	8	30 [22;39]	-	72 [61;81]	81 [71;88]	86 [76;93]	92 [83;97]	93 [86;96]	83 [69;91]	84 [75;90]	26 [17;38]
4	8	10 [6;16]	28 [19;39]	-	62 [53;70]	71 [61;80]	82 [71;89]	84 [74;90]	81 [69;89]	78 [63;88]	18 [10;28]
8	8	6 [2;17]	19 [12;29]	38 [30;47]	-	61 [51;71]	65 [51;76]	73 [62;82]	74 [59;85]	64 [55;73]	12 [3;34]
16	8	14 [6;28]	14 [7;24]	29 [20;39]	39 [29;49]	-	52 [41;63]	61 [50;71]	52 [41;64]	41 [32;51]	5 [1;25]
32	8	2 [0;9]	8 [3;17]	18 [11;29]	35 [24;49]	48 [37;59]	-	52 [42;63]	44 [32;57]	26 [17;36]	0 [0;30]
40	8	2 [1;8]	8 [4;14]	16 [10;26]	27 [18;38]	39 [29;50]	48 [37;58]	-	43 [30;56]	41 [26;58]	4 [1;18]
40	4	0 [0;24]	17 [9;31]	19 [11;31]	26 [15;41]	48 [36;59]	56 [43;68]	57 [44;70]	-	29 [18;41]	2 [0;11]
40	2	4 [2;9]	16 [10;25]	22 [12;37]	36 [27;45]	59 [49;68]	74 [64;83]	59 [42;74]	71 [59;82]	-	5 [1;17]
40	1	62 [48;75]	74 [62;83]	82 [72;90]	88 [66;97]	95 [75;99]	100 [70;100]	96 [82;99]	98 [89;100]	95 [83;99]	-

95% Agresti–Coull confidence intervals in grey. Each program played with 2 s per move; *komi* was 7.5 in all games.

© 2016 Macmillan Publishers Limited. All rights reserved

扩展数据表 10 | 单机可扩展性研究中各程序胜率的交叉表（百分比）

线程		1	2	4	8	16	32	40	40	40	40
	GPU	8	8	8	8	8	8	8	4	2	1
1	8	-	70	90	94	86	98	98	100	96	38
2	8	30	-	72	81	86	92	93	83	84	26
4	8	10	28	-	62	71	82	84	81	78	18
8	8	6	19	38	-	61	65	73	74	64	12
16	8	14	14	29	39	-	52	61	52	41	5
32	8	2	8	18	35	48	-	52	44	26	0
40	8	2	8	16	27	39	48	-	43	41	4
40	4	0	17	19	26	48	56	57	-	29	2
40	2	4	16	22	36	59	74	59	71	-	5
40	1	62	74	82	88	95	100	96	98	95	-

图中灰色部分为 95% Agresti-Coull 置信区间。每步棋的下法时间为 2 秒；所有对局的贴目数均为 7.5。

© 2016 麦克米伦出版有限公司。版权所有。

Extended Data Table 11 | Cross-table of win rates in per cent between programs in the distributed scalability study

Threads			40	12	24	40	64
	GPU		8	64	112	176	280
		CPU	48	428	764	1202	1920
40	8	48	-	52 [43; 61]	68 [59; 76]	77 [70; 82]	81 [65; 91]
12	64	428	48 [39; 57]	-	64 [54; 73]	62 [41; 79]	83 [55; 95]
24	112	764	32 [24; 41]	36 [27; 46]	-	36 [20; 57]	60 [51; 69]
40	176	1202	23 [18; 30]	38 [21; 59]	64 [43; 80]	-	53 [39; 67]
64	280	1920	19 [9; 35]	17 [5; 45]	40 [31; 49]	47 [33; 61]	-

95% Agresti–Coull confidence intervals in grey. Each program played with 2 s per move; *komi* was 7.5 in all games.

© 2016 Macmillan Publishers Limited. All rights reserved

扩展数据表 11 | 分布式可扩展性研究中各程序胜率的交叉表（百分比）

线程	GPU	中央处理器	40	12	24	40	64
			8	64	112	176	280
			48	428	764	1202	1920
40	8	48	-	52 68 77 81			
12	64	428 48		-	64 62 83		
24	112	764 32 36			-	36 60	
40	176	1202 23 38 64				-	53
64	280	1920 19		17	40 47		-

图中灰色部分为 95% Agresti-Coull 置信区间。每步棋的下法时间为 2 秒；所有对局的贴目数均为 7.5。

© 2016 麦克米伦出版有限公司。版权所有。